

Almacenamiento físico e índices



Base de datos

Facultad de Ingeniería y Ciencias Agrarias

Temas



Persistencia de datos



Unidad de almacenamiento



Organización de los archivos



Los managers de la Arquitectura



Volumetría



Tipos de datos



Tamaño de las tablas y de la BD



Índices



Índices. Clasificación



Otros tipos de índices

Persistencia de datos

Los DBMS necesitan almacenar grandes cantidades de datos.

¿Cómo persisten esos datos? ¿En disco? **SI (generalmente)**

¿Y cuándo no? Cuando utilizan almacenamiento **RAW**, pueden tener otras opciones.

Las capas lógicas que tiene que atravesar un usuario de base de datos hasta llegar al almacenamiento físico son las siguientes:



Unidad de almacenamiento

Los datos se almacenan en dispositivos de almacenamiento externo tales como discos y cintas, y se traen a memoria principal cuando se necesiten para algún procesamiento.

La unidad de información leída o escrita en **disco** es una **página**.

El tamaño de una página es un parámetro del DBMS, los valores típicos son 4KB u 8KB.

En estas **páginas**, se pueden almacenar uno o más **registros** (o menos de uno)

Un **archivo** de registros agrupa muchas **páginas** de **registros**.

Cada registro se identifica con un **RowId**. - Unívoco a nivel de Base de datos

Coste de E/S de páginas => **MINIMIZAR**



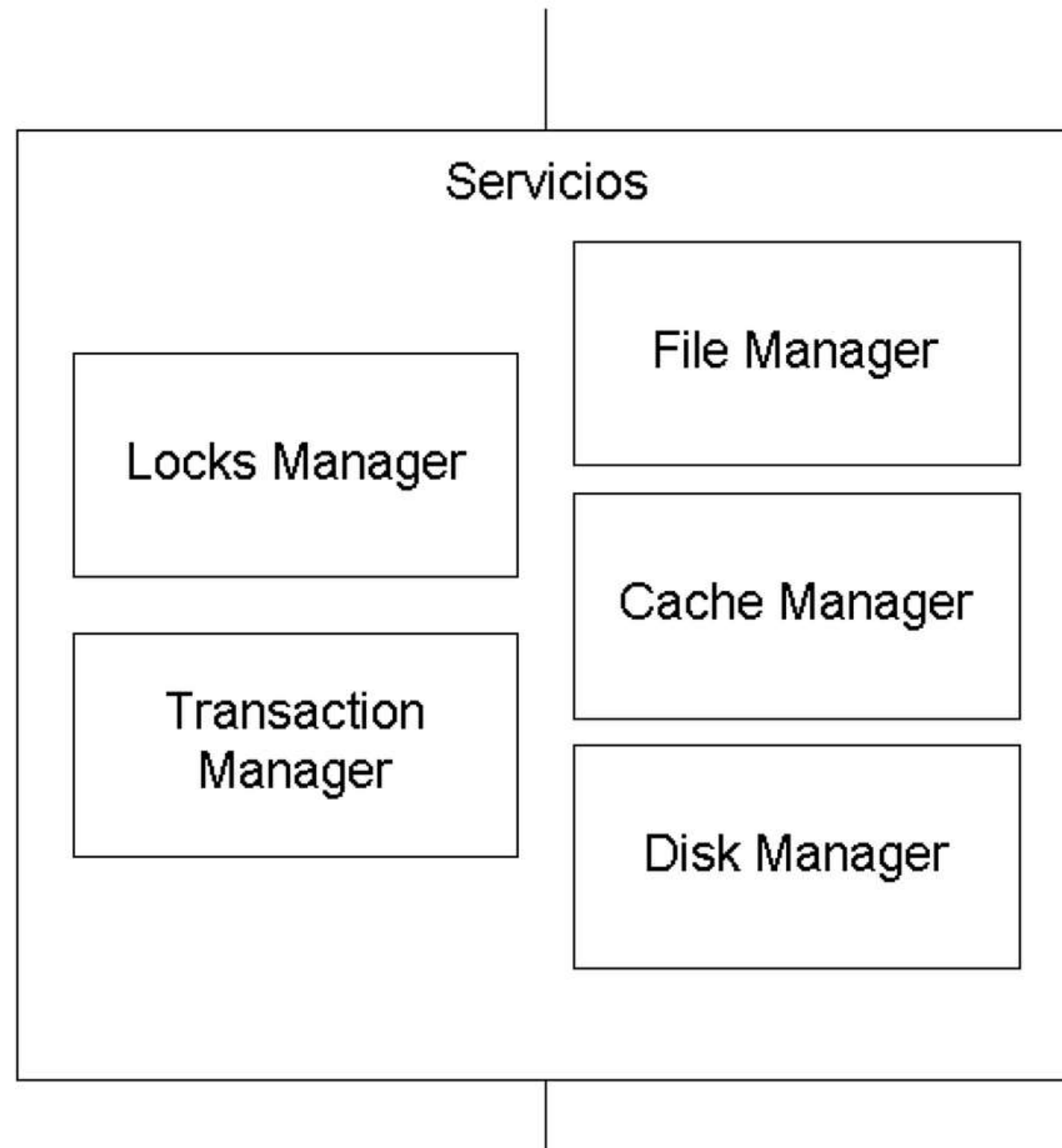
Organización de los archivos

La organización de los archivos es diferente en cada DBMS.

Archivos de registro secuenciales (o heap - montículo -): Los registros se almacenan en las páginas en el orden en que son insertados.

Archivos agrupados u ordenados: Los registros se almacenan en las páginas en un orden particular (por una relación de orden declarada sobre uno - o una lista - de atributos de la tabla en cuestión).

Los managers de la Arquitectura

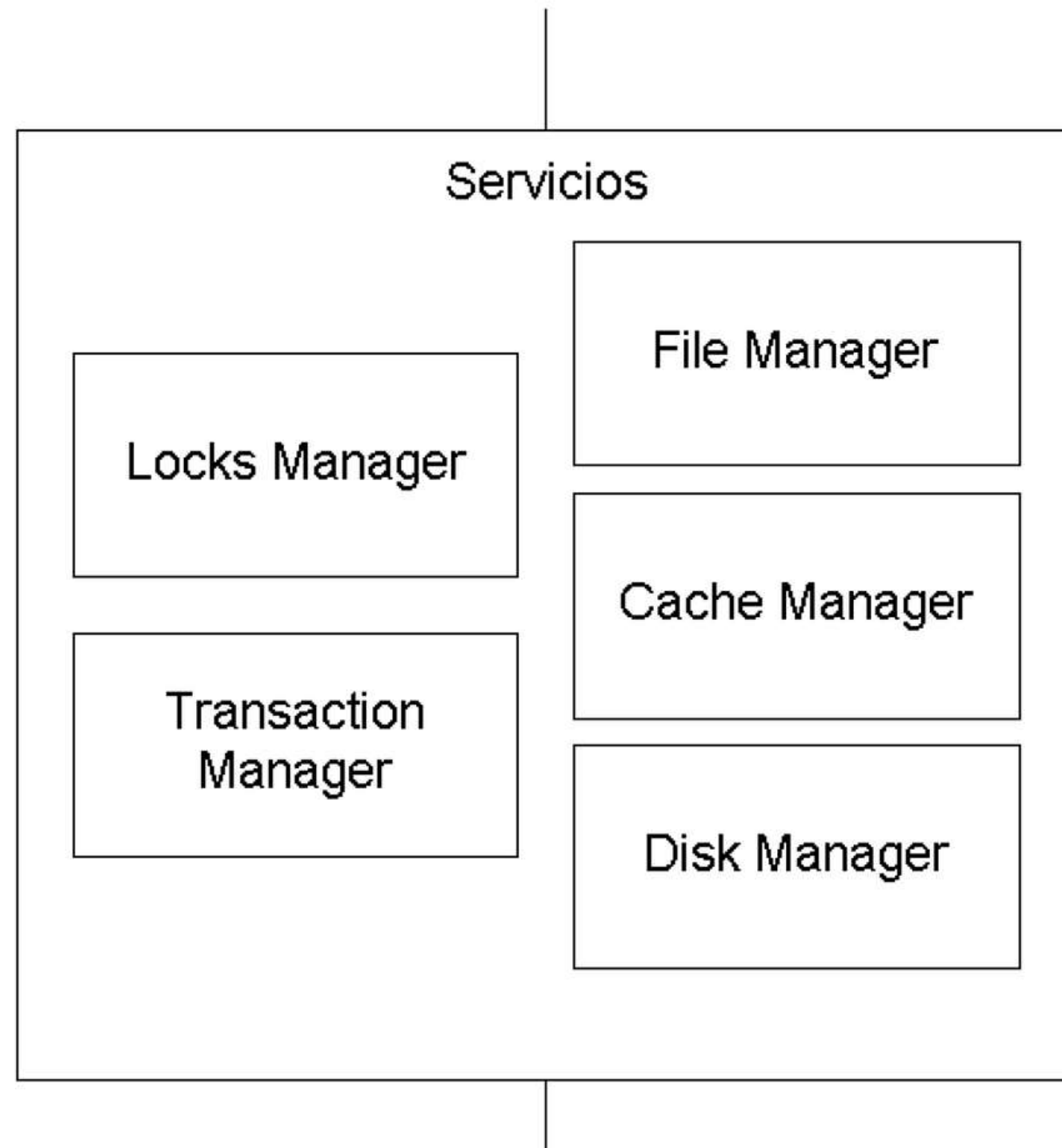


File Manager: Cuando necesita procesar un registro, lo pide al Cache Manager por RowID. Si no está en memoria, el CM lo traerá desde el disco.

Administra todos los archivos de la base de datos, las páginas que hay en c/u y el espacio libre en c/página de c/file.

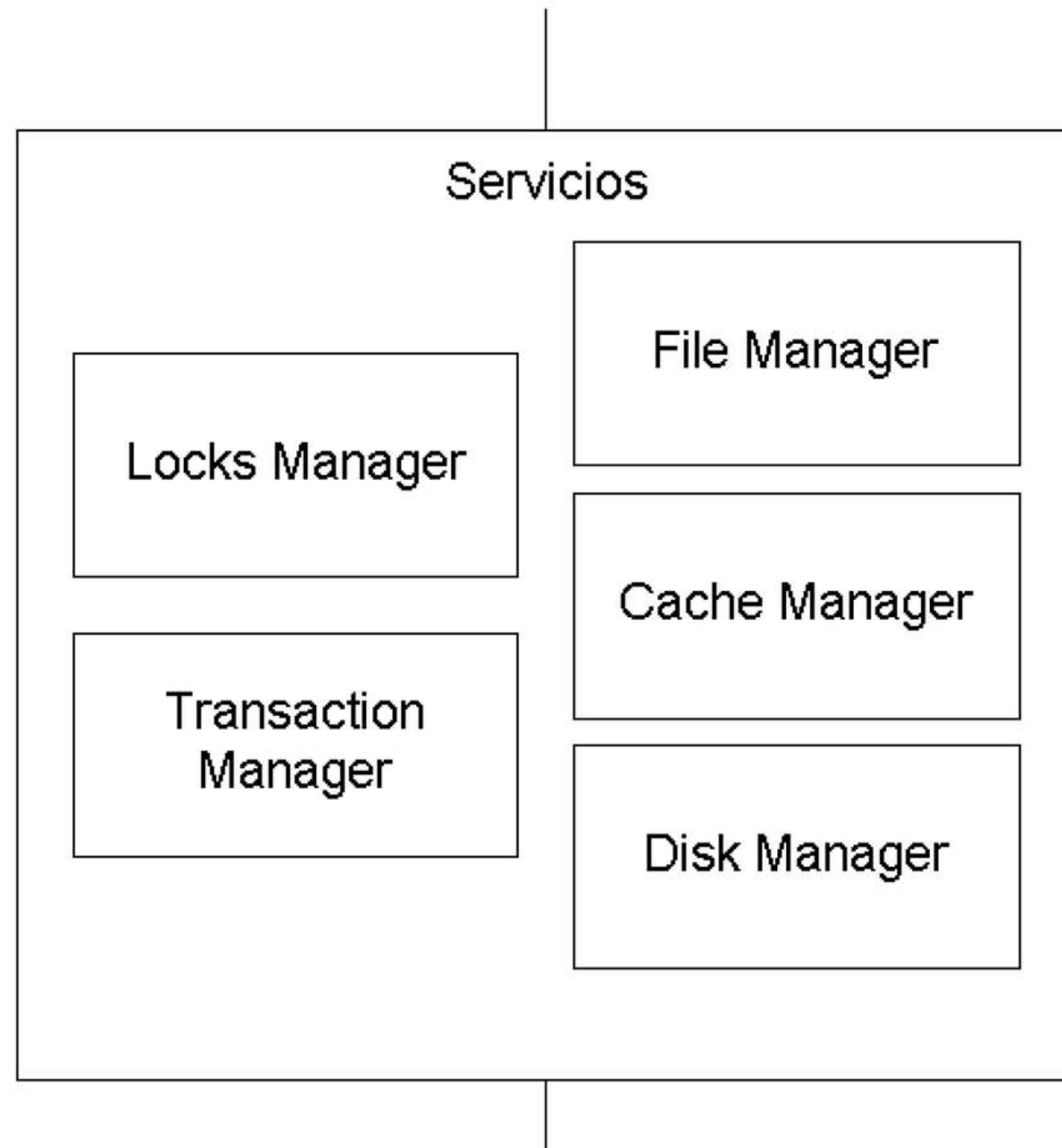
Crea y elimina archivos.

Los managers de la Arquitectura



Cache Manager: Gestiona espacio en memoria (RAM) para almacenar temporariamente los datos a procesar.

Los managers de la Arquitectura



Disk Manager: Gestiona el espacio libre y ocupado del disco.

Cuando el File Manager necesita espacio para un nuevo registro, el DM lo administra.

También lleva una traza de las páginas que están siendo utilizadas por el FM y reutilizará el espacio cuándo sea necesario.



Operaciones en archivos

Abrir: Localiza el archivo y genera un puntero al mismo, por el cual se puede acceder a las siguientes funciones.

Cerrar: desliga el archivo del proceso actual, y libera los recursos utilizados para utilizarlo.

Siguiente: Devuelve el siguiente registro (de tamaño y tipo especificado al llamar a la función), y mueve el puntero de archivo al siguiente.

Scan: que es en realidad un bucle de operaciones "siguiente", buscando el registro en particular.

BOF: Principio del archivo.

EOF: Fin del archivo.



Volumetría

Cuando se diseña una base de datos es necesario realizar una estimación del tamaño que tendrá la base de datos cuando esté llena. Esto ayudará a:

- Conseguir el rendimiento que necesitan las aplicaciones.
- Asegurar la cantidad física adecuada de espacio en disco necesario para almacenar los datos y los índices.
- Ayudar a determinar si el diseño de la base de datos necesita reajustes.

Estimación manual

- Tamaño de los tipos de datos.
- Tamaño de las tablas, sin considerar sus índices.
- Tamaño total de las tablas.

Tipos de datos

Fuertemente dependiente del RDBMS.

Longitud Fija:

CHAR [(size [BYTE | CHAR])] => La longitud que se le asigne

DATE => 7 Bytes

INTEGER => 8 Bytes

TIMESTAMP => Entre 7 y 11 bytes dependiendo la precisión.

Longitud Variable:

VARCHAR [(size [BYTE | CHAR])] => Como máximo la longitud que se le asigne, tiene un mínimo que es un parámetro del DBMS.

NUMBER [(p [, s])] => Entre 1 y 22 Bytes

FLOAT (p) => Entre 1 y 22 Bytes

BLOB => Máximo: $(4\text{Gb} - 1\text{b}) * (\text{tamaño del bloque de la base de datos})$.

Tamaño de las tablas, sin considerar sus índices

Define:

Num_Rows = número de filas de la tabla

Num_Cols = número total de columnas (de longitud fija y variable)

Fixed_Data_Size = tamaño total en bytes de todas las columnas de longitud fija

Num_Variable_Cols = número de columnas de longitud variable

Max_Var_Size = tamaño total máximo de bytes de todas las columnas de longitud variable

Se fija el **Page_Size** (tamaño de página). Por ejemplo 8096.

Tamaño de las tablas, sin considerar sus índices

Tamaño de los datos de longitud variable:

Si hay columnas variables:

$$\text{Variable_Data_Size} = 2 + (\text{Num_Variable_Cols} \times 2) + \text{Max_Var_Size}$$

Si no hay columnas de longitud variable:

$$\text{Variable_Data_Size} = 0$$

Tamaño total de la fila: $\text{Row_Size} = \text{Fixed_Data_Size} + \text{Variable_Data_Size} + \text{Num_Cols}$

Filas por página: $\text{Rows_Per_Page} = \text{Page_Size} / (\text{Row_Size} + 2)$

Filas libres por página:

$$\text{Free_Rows_Per_Page} = \text{Page_Size} \times ((100 - \text{Fill_Factor}) / 100) / \text{Row_Size}$$

Se calcula el número de páginas necesarias para almacenar todas las filas:

$$\text{Num_Pages} = \text{Num_Rows} / (\text{Rows_Per_Page} - \text{Free_Rows_Per_Page})$$

$$\text{Tamaño de la tabla (bytes)} = \text{Page_Size} \times \text{Num_Pages}$$



Índices. Ejemplo

Supongamos que tenemos un registro de empleados, cada uno de ellos con los campos **edad, nombre y sueldo**.

Si deseo recuperar los registros de empleados en orden creciente de edad, una buena organización del archivo es ordenarlo de esa forma, en orden creciente de edad. Pero este orden es de mantenimiento costoso si el archivo se modifica frecuentemente.

Y si quisiera determinar, por ejemplo, todos los empleados que ganen más de \$150.000, sería necesario recorrer todo el archivo para encontrar los registros.



Índices

Para reducir la complejidad de estas búsquedas sobre los archivos de datos, se plantea mantener estructuras auxiliares que, guardando sólo el atributo de búsqueda, un identificador del ítem y un puntero a su posición en disco, permita obtener, sin recorrer el archivo de datos, la posición de el/los ítems de datos buscados.

Los índices son estructuras almacenadas en disco:

- creadas para poder ser utilizadas en memoria principal.
- diseñadas para que el índice, o la parte que se supone que se utilizará esté en memoria, evitando accesos I/O a disco.



Índices. Clasificación

EN CUANTO A LA CANTIDAD DE ENTRADAS O VALORES

- **Denso:** Hay igual cantidad de entradas en el índice que en el archivo de datos. Esto suele pasar cuando el índice está armado para la clave primaria de la tabla.
- **No denso:** menor al archivo de datos.

EN CUANTO AL ORDEN

- **Agrupados:** Cuando el orden en el cual están dispuestos los archivos de datos es el del atributo por el cual se construyó el índice. Ambas estructuras estarían ordenadas de igual manera.
- **No agrupados:** en caso contrario.

Índices. Clasificación

EN CUANTO AL NIVEL

- Índices de un nivel

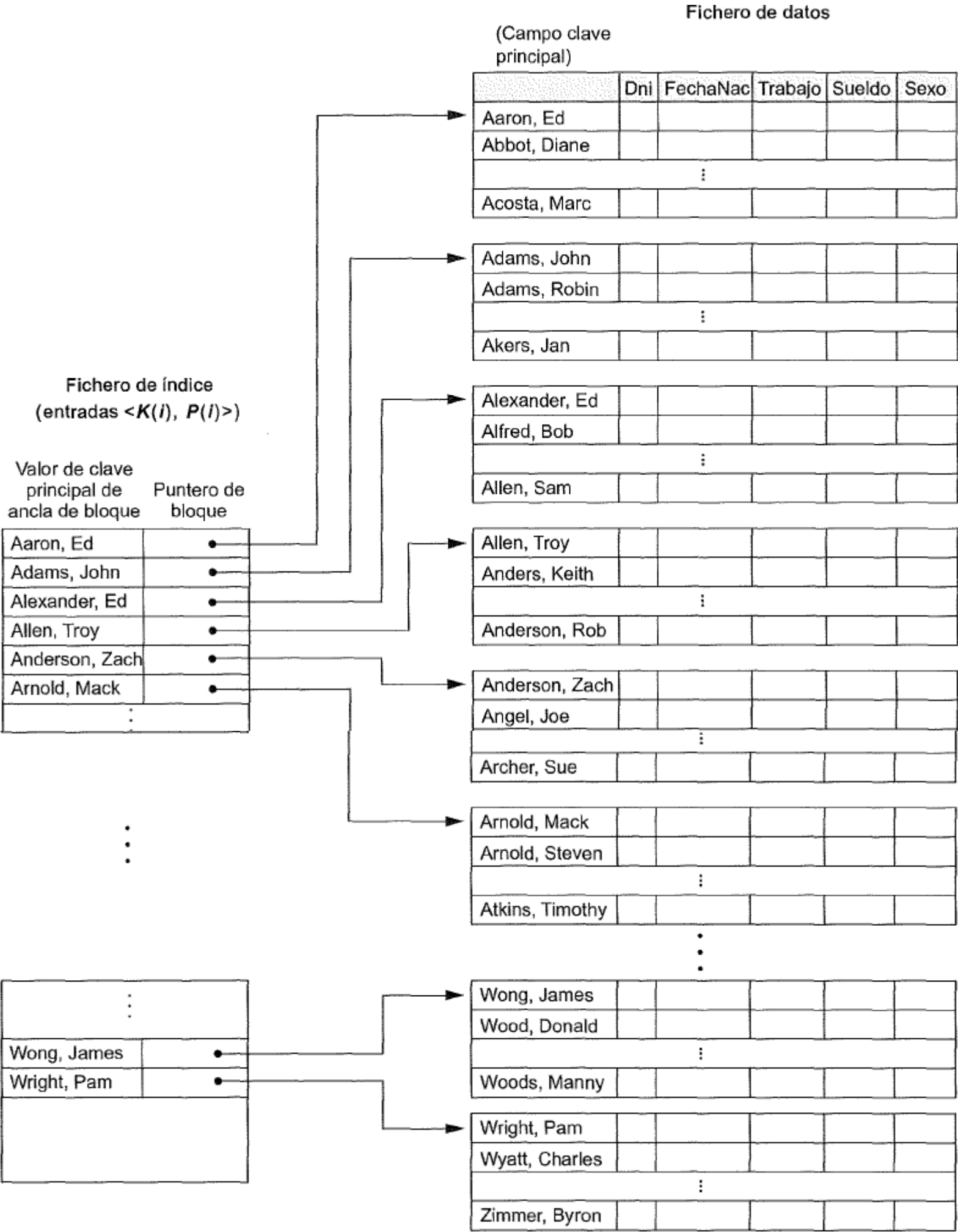
- **Primario:** Se trata de un archivo ordenado, con celdas de tipo (id, puntero al registro).

El número de entradas es igual al número de bloques que contienen registros, por lo que es un tipo de índice no denso.

La inserción es costosa: esto se debe a que, siendo el índice una estructura lineal ordenada, una inserción en medio de la misma implica mover a todos los elementos que le siguen. Sucede lo mismo con el archivo de datos de este tipo.

Índice Primario. Ejemplo

El índice principal en el campo clave de ordenación del fichero



Índices. Clasificación

EN CUANTO AL NIVEL (cont.)

- Índices de un nivel (cont.)

2. **Agrupamiento:** Ordenados por campo no clave; es decir, el atributo del índice no es la clave principal de la tabla. Muchos registros tienen el mismo valor de clave, ya que la idea del índice es la de poder ejecutar búsquedas por valor de atributo.

Archivo ordenado, con celdas (campo, puntero).

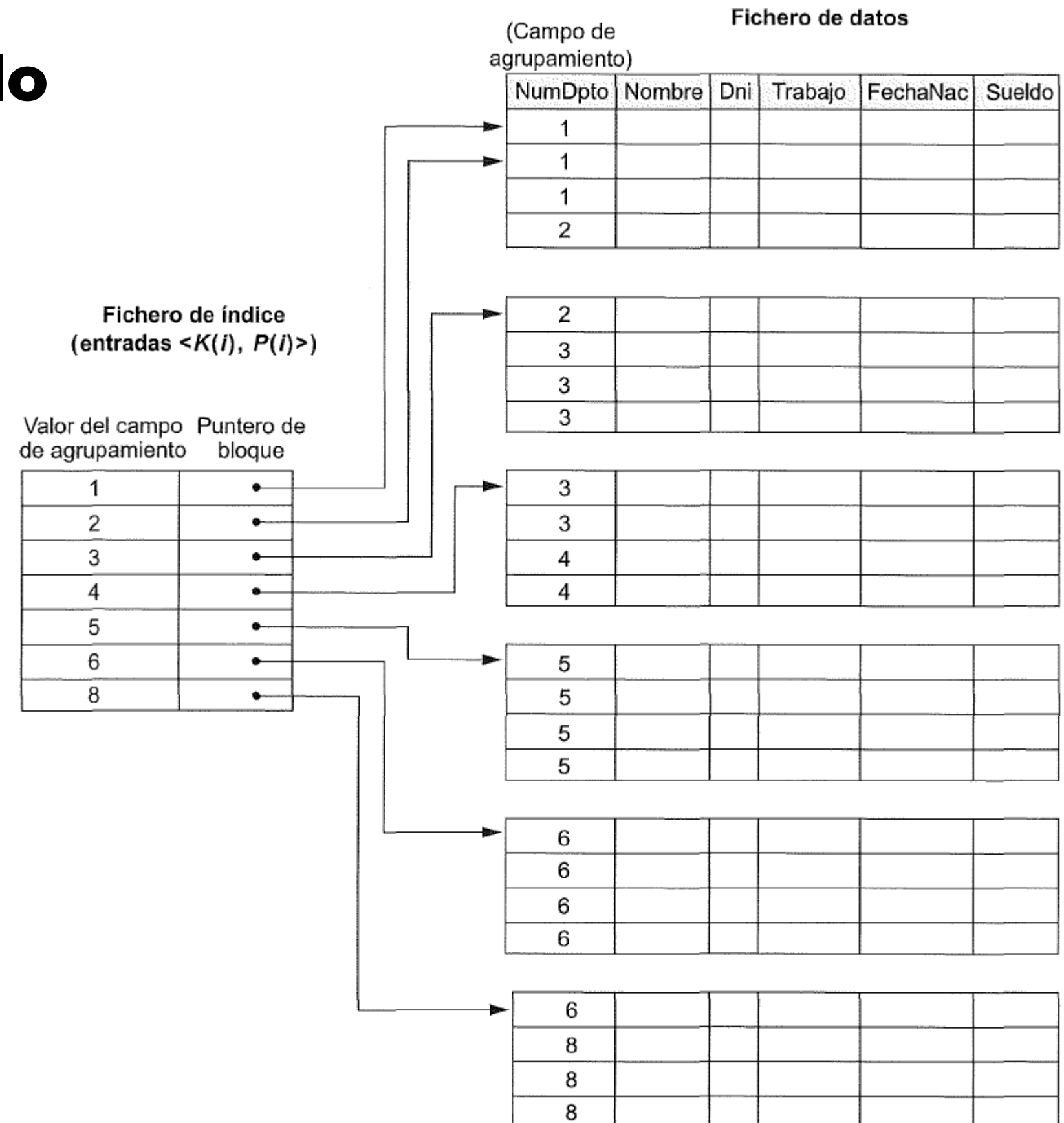
Por cada valor almacenado, hay un bloque en disco.

No denso.

Es similar en forma al Hashing

Índice Agrupamiento. Ejemplo

Un índice agrupado en el campo no clave de ordenación NumDpto de un fichero EMPLEADO



Índices. Clasificación

EN CUANTO AL NIVEL (cont.)

- Índices de un nivel (cont.)

3. **Secundario:** Archivo ordenado, con celdas (campo de no ordenamiento, bloque/registro).

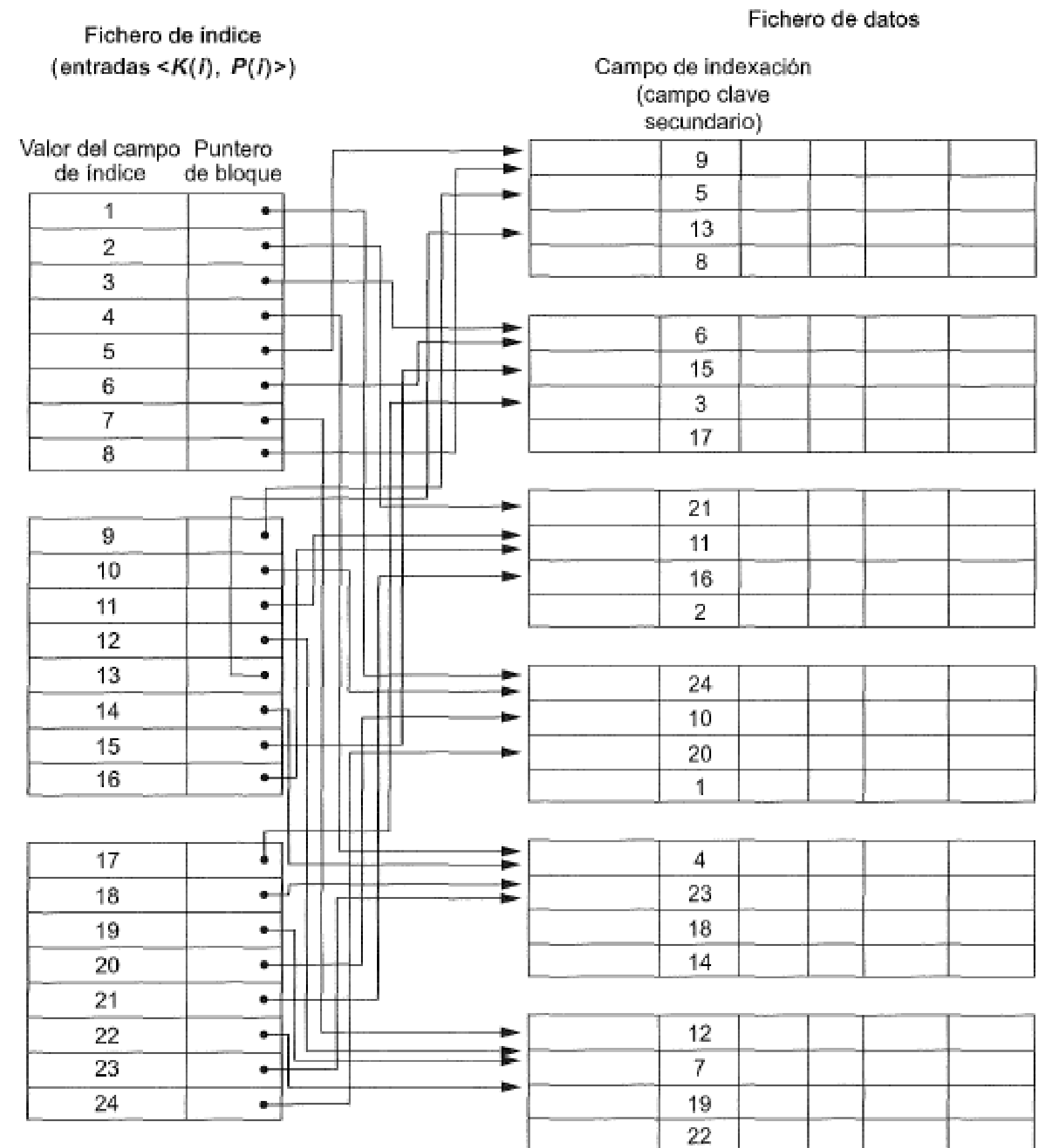
Si el índice se construye por campo clave, es denso.

Si se lo construye por campo no clave, puede ser de tres formas distintas:

- 1 clave y n entradas, lo que hace al índice denso
- 1 clave y una lista variable con los links a los datos, lo que hace al índice no denso
- Nivel de indirección (el elemento del índice apunta a un contenedor con links a los datos).

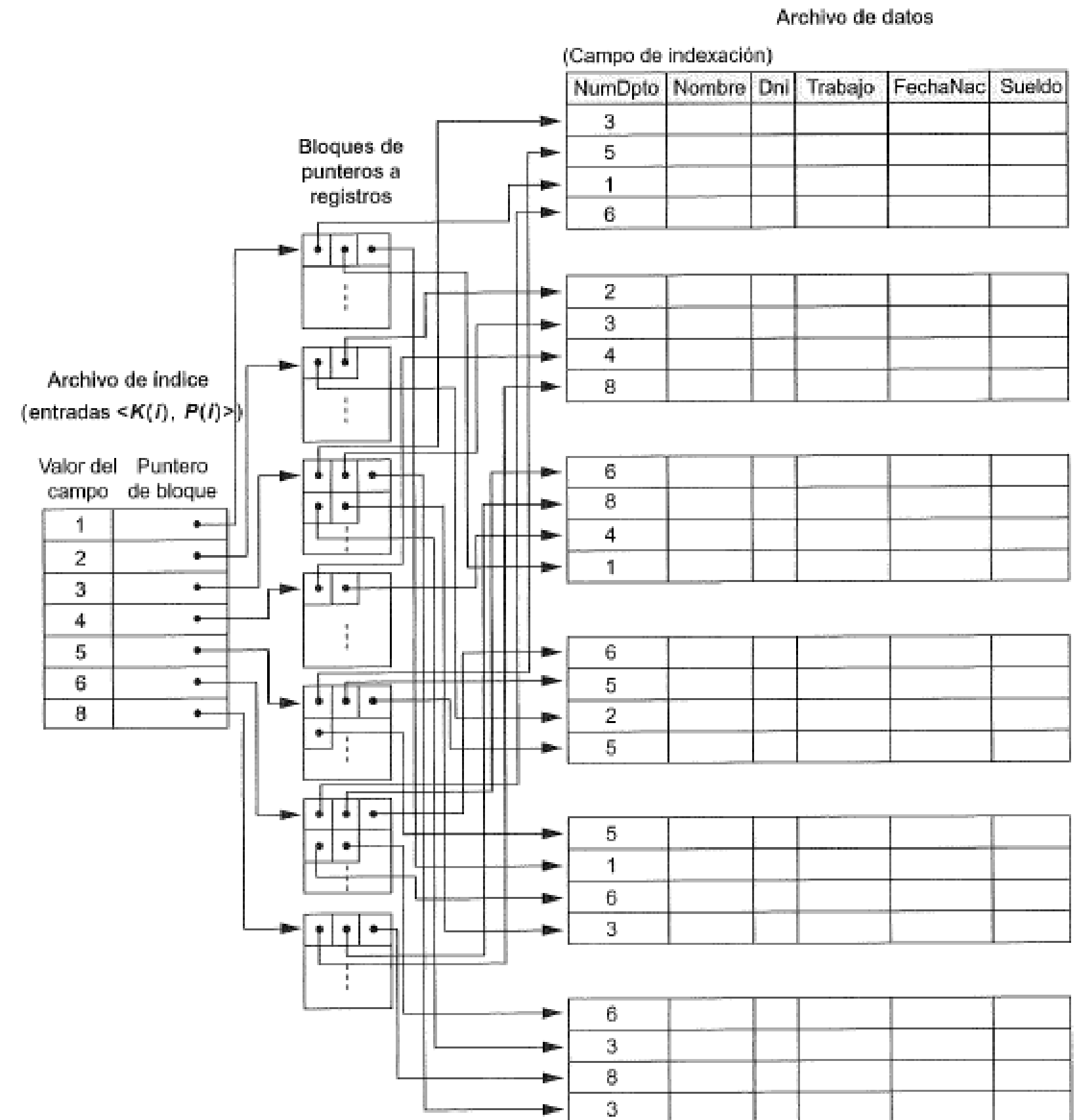
Índice Secundario. Ejemplos

Índice secundario denso (con punteros de bloque) en un campo clave desordenado de un fichero.



Índice Secundario. Ejemplos

Índice secundario (con punteros de registro) en un campo que no es índice e implementado utilizando un nivel de indirección, de modo que las entradas del índice son de longitud fija y tienen valores de campo únicos.



Preguntas

¿Cuántos índices tipo clúster (agrupados) puede tener una tabla?

- Los índices clúster ordenan y almacenan las filas de los datos de la tabla o vista de acuerdo con los valores de la clave del índice. Son columnas incluidas en la definición del índice. Solo puede haber un índice clúster por cada tabla, porque las filas de datos solo pueden estar almacenadas de una forma.

¿Cuántos índices secundarios puede tener una tabla?

- No tengo límites
-

Índices. Clasificación

EN CUANTO AL NIVEL (cont.)

- Índices de múltiples niveles
- **Árboles B:** Son estructuras más dinámicas, que permiten operar con inserciones o eliminaciones más fácilmente. Todos los nodos tienen punteros que referencian al archivo de datos.
- **Árboles B+:** Son árboles B en dónde sólo los nodos de tipo hoja tienen enlaces a páginas de datos. Si el campo es clave, apunta a un bloque/registros. Si no, bloques con apuntadores a los datos.

Índice de multiples niveles: Árboles. Ejemplo

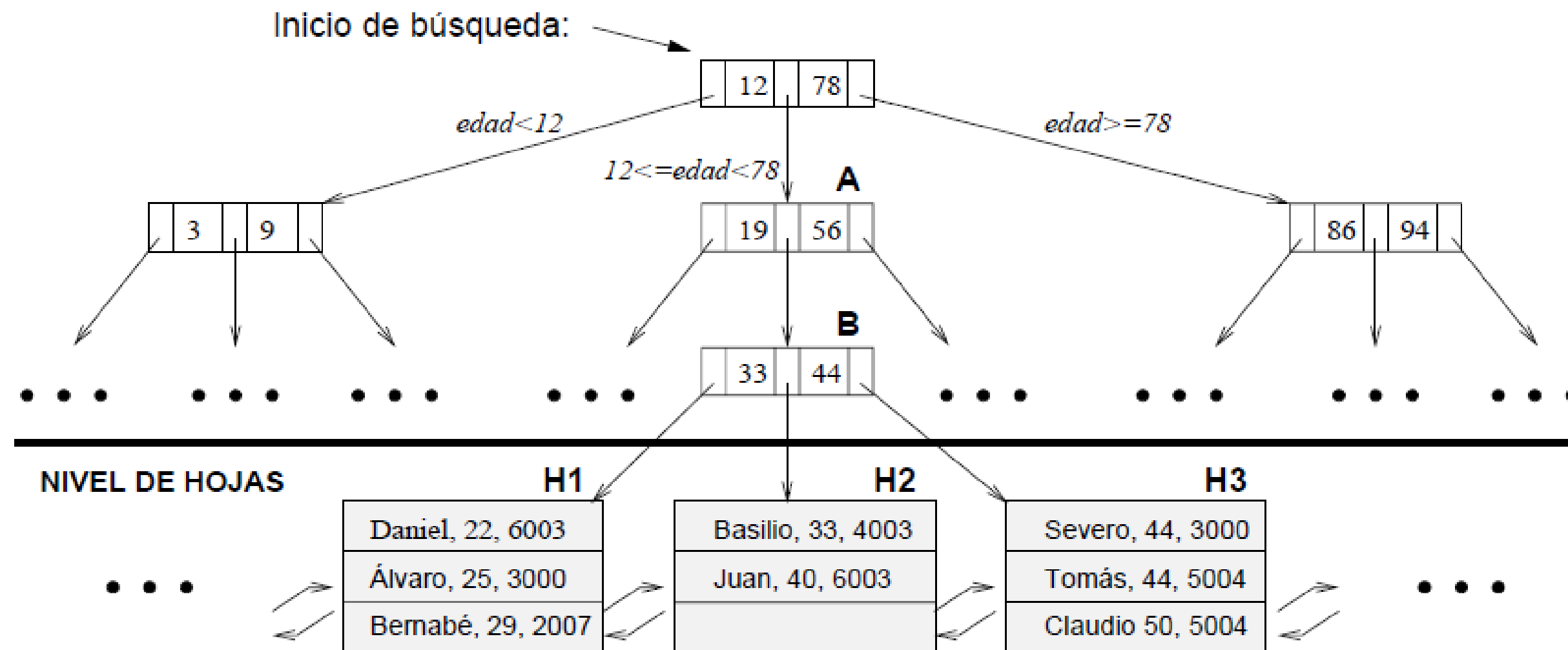


Figura 9.3 Índice estructurado en árbol



Otros índices

Bitmaps:

Arrays con cadenas de bits que representan, para cada ítem de datos, cuál es el valor del dominio de la clave por la cual se hizo el índice que tiene el atributo.

Suele recomendarse utilizarlo para atributos cuyo dominio tenga una cardinalidad baja

No tienen buen comportamiento en inserciones/eliminaciones.



M	F
1	0
1	0
0	1
1	0

<i>idcli</i>	<i>nombre</i>	<i>sexo</i>	<i>categoría</i>
112	José	M	3
115	Ramón	M	5
119	Susana	F	5
112	Guillermo	M	4

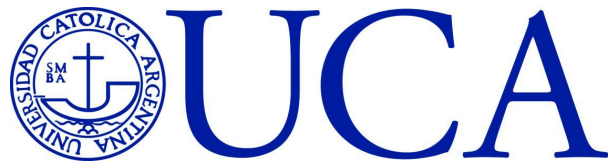
1	2	3	4	5
0	0	1	0	0
0	0	0	0	1
0	0	0	0	1
0	0	0	1	0

Figura 16.9 Índices de mapas de bits de la relación Clientes



Preguntas





Bases de Datos I

Clase 4

Transacciones y Concurrency

Transacción

Una transacción consiste en una **interacción** con una estructura de datos que, aun siendo compleja y estando compuesta por varios procesos que se han de aplicar uno después del otro, se quiere que sea equivalente a una interacción **atómica**. Es decir, que se realice de una sola vez y que la estructura a medio manipular no sea jamás alcanzable por el resto del sistema.

Ejemplo

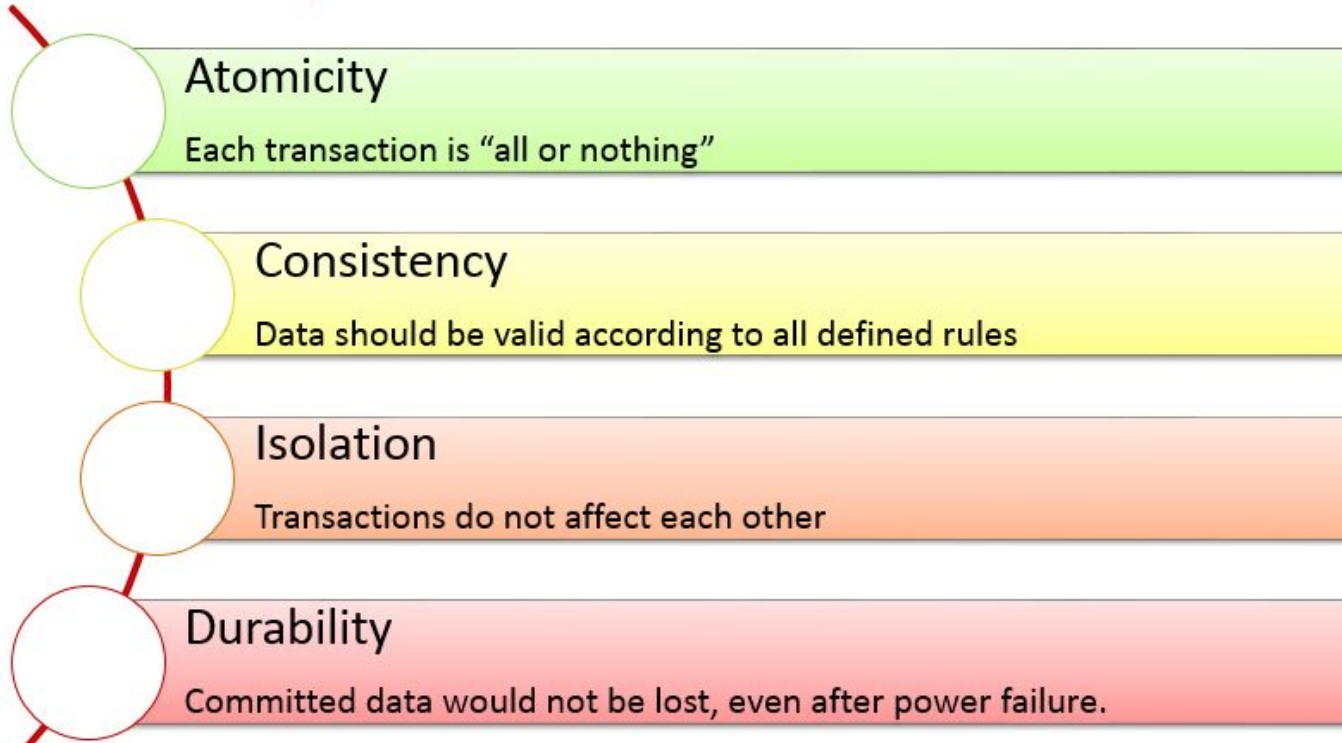
- Transferencia bancaria
- Transferir \$5000 de la cuenta A a la B:
- Saldos:
 - A: \$20.000
 - B: \$
- Pasos lógicos
 - Comprobar si en la cuenta A hay dinero suficiente.
 - Restar \$5000 de la cuenta de A
 - con lo que su saldo pasa a ser de \$15000.
 - Sumar \$5000 a la cuenta de B,
 - con lo que los saldos quedan A=\$15000 y B=\$5000

Posibles problemas con el ejemplo

- Si entre el paso 2 y el 3 el sistema sufre un error inesperado:
 - las cuentas quedarían como
 - $A=15000$
 - $B=0$
 - con lo cual... Se han volatilizado \$5000....
- Este ejemplo ilustra por qué las transacciones tienen un comportamiento deseado de Todo o nada, o se realiza completamente o no debe tener ningún efecto.

Propiedades de las transacciones

ACID Properties



Atomicidad

- Atómica:
 - todas las acciones (dentro de la transacción en cuestión) se completan exitosamente o ninguna de ellas se completa, pero nunca puede quedar un estado intermedio, de modo que el usuario **no debe preocuparse** por el efecto de transacciones incompletas.
 - Se garantiza que ninguna transacción puede quedar "**por la mitad**", aunque sea removiendo o volviendo atrás los cambios que quedaron "a medio hacer",
 - Para esto, la base de datos mantiene un registro de las acciones que se registran sobre la misma, llamado Log (bitácora).

Consistencia

- Cada transacción debe preservar la consistencia de la base de datos.
- Una transacción siempre comienza en una instancia de base de datos consistente, por más que se encuentre (el DBMS) en "el medio" de la ejecución de otra transacción.

Aislamiento

- Una transacción corre "sola" sin considerar el ruido que pueda hacer otra transacción ejecutándose en el mismo momento:
 - las transacciones corren aisladas,
 - o protegidas de los efectos de modificaciones concurrentes.
- La aislación de las transacciones se asegura en un DBMS de modo que ejecutar n transacciones (quizás simultáneamente) tenga el mismo efecto que ejecutar una por una de las n anteriores.
- Por ejemplo, existen 2 transacciones simultáneas llamadas T1 y T2,
 - el efecto global en la base de datos de ejecutar las 2 transacciones concurrentemente es el mismo que el de ejecutar T1 seguido de T2 o T2 seguido de T1 (el motor de base de datos no asegura cuál es la transacción que se ejecuta primero, pero **sí asegura** que existe un primero y un segundo).

Durabilidad

- Una vez que el DBMS confirma que la transacción ha sido exitosa:
 - sus efectos han sido persistidos (incluso si hay una falla del sistema).
 - El log que la base de datos mantiene de las escrituras que se hacen es utilizado también para garantizar la durabilidad de los datos, es decir para "recordar" lo que no fue persistido (quizás debido a una falla de hardware o de energía eléctrica que bajó el sistema).

Schedules

- Una transacción es vista por una base de datos como una serie de acciones.
- Conjunto de acciones parcialmente ordenados:
 - ya que el orden relativo de ciertas acciones puede no ser importante.
- Toda transacción tiene un final claro:
 - no siempre explícito
 - se puede decir que la transacción termina bien y por consecuencia todos los cambios a la base de datos son finalizados, este caso se conoce como "commit".
 - En cambio, si explícitamente o no (sea por un error en la acción a realizar programática o por una falla de hardware) se habla de un "abort" o de un "rollback".
- Se define un Schedule cómo una lista de acciones (pueden ser lecturas, escrituras, commits o rollbacks) de un conjunto de transacciones, y el orden en el que dos acciones de una transacción T está en el schedule debe ser el mismo que el orden en el que aparece en T.

Schedules

- Un Schedule que posee un commit o abort por cada transacción que incluye se llama Schedule Completo:
 - Incluye todas las acciones de las transacciones en cuestión.
- Si el orden de ejecución de las acciones de cada transacción en el schedule es lineal, es decir, no hay acciones de distintas transacciones mezcladas, se habla de un schedule serial

Anomalías asociadas a ejecuciones entrelazadas



Lectura sucia

- Aquí una transacción T2 lee datos que todavía no han sido confirmados por una transacción T1.
- Nótese que una transacción debe dejar una base de datos consistente al final de su ejecución y debe encontrarla consistente antes de comenzar su ejecución, pero no se garantiza nada en el medio de la ejecución de la misma.

T1	T2
R(a)	-
W(a)	-
-	R(a)
-	W(a)
-	R(b)
-	W(b)
-	Commit
R(b)	-
W(b)	-
Commit	-

Lectura irrepetible

- Suponemos que una transacción T2 puede cambiar el valor de **a** que ya ha sido leída por T1 mientras T1 está en progreso.
- Esta situación causa dos problemas:
 - Si T1 trata de leer el valor de **a** nuevamente, será distinto, lo cual no ocurriría en una ejecución serial, por eso esta situación se llama lectura irrepetible.
 - Suponiendo que T1 y T2 leyeron el mismo valor de **a**, digamos 5, y entonces T1 quiere incrementar **a** en 1, cambiándolo a 6, y T2 quiere decrementar **a** en 1, decrementa el valor que encontró (5), dejándolo en 4,
 - Corriendo estas dos transacciones en forma serial, el resultado sería siempre 5.

Overwriting Uncommitted Data

- T2 pisa el valor de a, que sido ya modificado por T1 (en progreso),
- Suponiendo que en la tabla de empleados tenemos dos registros (representando 2 empleados) que deben mantener sus salarios iguales.
 - T1 setea sus salarios a \$3000
 - T2 los setea en \$4000,
 - si se ejecutan transacciones seriales:
 - o ambos quedan con un salario de \$3000 o de \$4000
 - Si se ejecutan en forma entrelazada, puede pasar que:
 - T1 pone \$3000 a empleado1
 - T2 pone \$4000 a empleado2
 - T1 pone \$3000 a empleado2
 - T2 pone \$4000 a empleado1
 - Como resultado: empleado1 tiene un salario de \$4000 y empleado2, de \$3000, lo cual es inconsistente en el modelo que imaginamos.

Strict 2 phase locking (Strict 2PL)

- Si una transacción quiere leer un objeto:
 - debe primero pedir un shared lock sobre el objeto pertinente.
- Si una transacción quiere escribir un objeto,
 - debe primero pedir un exclusive lock sobre el objeto pertinente.
- Todos los locks retenidos por una transacción son liberados cuando la transacción se completa (commit o rollback)
- Cumpliendo estas reglas, un DBMS puede entrelazar acciones entre transacciones de forma segura.

T1	T2
R(a)	-
R(c)	-
W(c)	-
Commit	-
-	R(a)
-	R(b)
-	W(b)
-	Commit

T1	T2
S(a)	-
R(a)	-
-	S(a)
-	R(a)
-	X(b)
-	R(b)
-	W(b)
-	Commit
X(c)	-
R(c)	-
W(c)	-
Commit	-

Deadlock

- Ocurre deadlock cuándo dos transacciones simultáneas están esperando que la otra libere su candado sobre un elemento.

T1	T2
S(a)	S(b)
R(a)	R(b)
X(b)	X(a)
Free(a)	Free(b)
R(b)	R(a)
W(b)	W(a)
Commit	Commit

Deadlock

- Una posibilidad para evitar el deadlock es utilizar la variante conservadora del S2PL, pero evidentemente se estaría limitando la concurrencia.
- Otro protocolo de prevención de deadlock que han utilizado implementaciones arcaicas, consiste en ordenar los recursos de la base de datos de forma total y hacer uso dentro de las transacciones en ese orden, obligando al programador a conocer ese orden...

Deadlock

- Otra alternativa consiste en etiquetar con timestamps las transacciones.
- Suponiendo que hay 2 transacciones que esperan por recursos cruzados (como en la figura arriba), las reglas que se pueden utilizar son:
 - esperar-morir: Una transacción más antigua puede esperar a que termine una más nueva, pero una transacción más reciente que solicite un elemento bloqueado por una más antigua se abortará y reiniciará.
 - herir-esperar: Se hace lo contrario, una transacción más reciente puede esperar a que termine una más antigua, pero una transacción más antigua que solicite un elemento bloqueado por la otra, desalojará a la más reciente abortándola.
- En última instancia, las dos reglas hacen lo mismo: que aborte la más reciente.
- Técnicas son libres de deadlocks
- Pero hacen que muchas transacciones se aborten y reinicien.

Deadlock

- Otras técnicas de concurrencia no necesitan utilizar timestamps, como por ejemplo:
 - No Waiting (NW):
 - Si una transacción no puede obtener un candado, se aborta de inmediato y se reinicia después de cierto tiempo aleatorio sin comprobar si ocurrirá o no un deadlock. Obviamente este enfoque puede hacer que muchas transacciones se aborten innecesariamente.
 - Cautious Waiting (CW):
 - Es una variación de NW, pero que intenta evitar la cantidad de transacciones abortadas innecesariamente. Suponiendo que la transacción T_i intenta bloquear un elemento X, pero no puede, dado que el elemento X está siendo bloqueado por una transacción T_j con un candado en conflicto
 - Si T_j no está detenida (esperando un elemento bloqueado por otra transacción), entonces se detiene T_i , la cual puede esperar, de lo contrario se aborta T_i .

Deadlock

- Otra técnica consiste en utilizar tiempos predefinidos a fin de abortar las transacciones que esperen por un bloqueo de otra transacción más de ese tiempo pre estipulado (time out).
- Otro enfoque distinto, consiste en detectar los deadlocks, en la cual el DBMS verifica periódicamente si el motor está en un estado de deadlock.
 - no es muy útil cuando la carga transaccional del DBMS es muy alta
 - o la longitud de las transacciones es muy larga.
 - No es recomendado para sistemas OLTP.

Deadlock

- Una forma sencilla de detectarlos es construyendo un grafo de espera:
 - Se crea un nodo por cada transacción del schedule (plan)
 - Se crea un arco de T_i a T_j si la T_i está esperando para bloquear un elemento X que está bloqueado momentáneamente por T_j .
 - Se elimina el arco anterior cuándo T_j libera el/los candado/s sobre el/los elementos que estaba esperando T_i .
 - Si en el grafo hay un ciclo, quiere decir que hay un estado de deadlock.
 - El problema de este enfoque consiste en determinar cuándo debe el DBMS determinar que hay o no deadlock.
 - Si el sistema determina que hay deadlock, determinará qué transacción o que transacciones abortará y reiniciará, a estas transacciones se las llama transacciones víctimas.
 - Aun así existen problemas que pueden ocurrir:
 - Reinicio cíclico
 - Espera indefinida
 - Inanición

Control de concurrencia basado en ordenamiento por TimeStamp

- Se asigna un timestamp a cada transacción dentro de un plan o schedule.
- El algoritmo debe asegurar que por cada elemento utilizado por más de una transacción en el plan, el orden en que se tiene acceso al elemento no viole la serializabilidad del schedule
- va a asociar a cada elemento X dos timestamp:
 - ReadTS(X): consiste en el timestamp más grande de todos los timestamp de las transacciones que han leído con éxito el elemento X.
 - WriteTS(X): consiste en el timestamp más grande de todos los timestamp de las transacciones que han escrito con éxito el elemento X.

Control de concurrencia basado en ordenamiento por TimeStamp

- Siempre que una transacción intente leer o escribir el elemento X, compara el timestamp de la transacción con el ReadTS y el WriteTS de X, para asegurar que no se viole el orden de ejecución.
- Si la transacción viola el orden, también viola el plan en serie equivalente, así que se aborta:
 - reintroduciéndose al sistema con un timestamp más nuevo.

Transacciones Distribuidas

- Las bases de datos distribuidas surgieron con la necesidad o la imposibilidad de administrar grandes cúmulos de datos de manera centralizada.
- Más complejo:
 - protocolos de replicación distribuida,
 - protocolos de confiabilidad distribuidos,
 - control de concurrencia distribuido
 - procesamiento y ejecución de consultas distribuidas.
- Coordinador de Transacciones Distribuidas

Transacciones Distribuidas: Ejemplo

- Mismo caso transferencia bancaria entra A y B
- Supongamos que una de las cuentas se encuentra en otra base de datos (incluso gestionada por otro RDBMS en una ubicación diferente a la primer cuenta).
- Los pasos lógicos sería:
 - Comprobar si en la cuenta A (DBMS1) hay dinero suficiente.
 - Restar \$5000 de la cuenta de A (DBMS1), con lo que su saldo pasa a ser de \$15000.
 - Sumar \$5000 a la cuenta de B (DBMS2), con lo que los saldos quedan A=\$15000 y B=\$5000

Componentes para el Procesamiento de Transacciones Distribuidas

- Estándar abierto: X/Open XA (X Open Distributed Transaction Processing Model). Dentro de ese modelo, los componentes son:
 - Application Program: La lógica de negocio desarrollada por los programadores.
 - Transaction Manager: Software que coordina las transacciones globales y asegura la sincronización de las bases de datos en los casos en que sea necesario.
 - Resource Manager: Software que administra parte de los recursos compartidos del sistema, tales como las bases de datos. Estos recursos no sólo pueden estar representados por DBMS, sino que pueden ser gestores de colas (JMS por ejemplo), middleware de comunicaciones, y otras piezas de software.
 - Communications Resource Manager: Software que sirve como un Resource Manager especializado para comunicar datos entre las aplicaciones.

Roles en Transacción distribuida

- Coordinador: Es el Transaction Manager que administra la transacción global.
- Participante: Es uno de los varios Resource Managers (DBMS generalmente) que son afectados por la transacción distribuida.

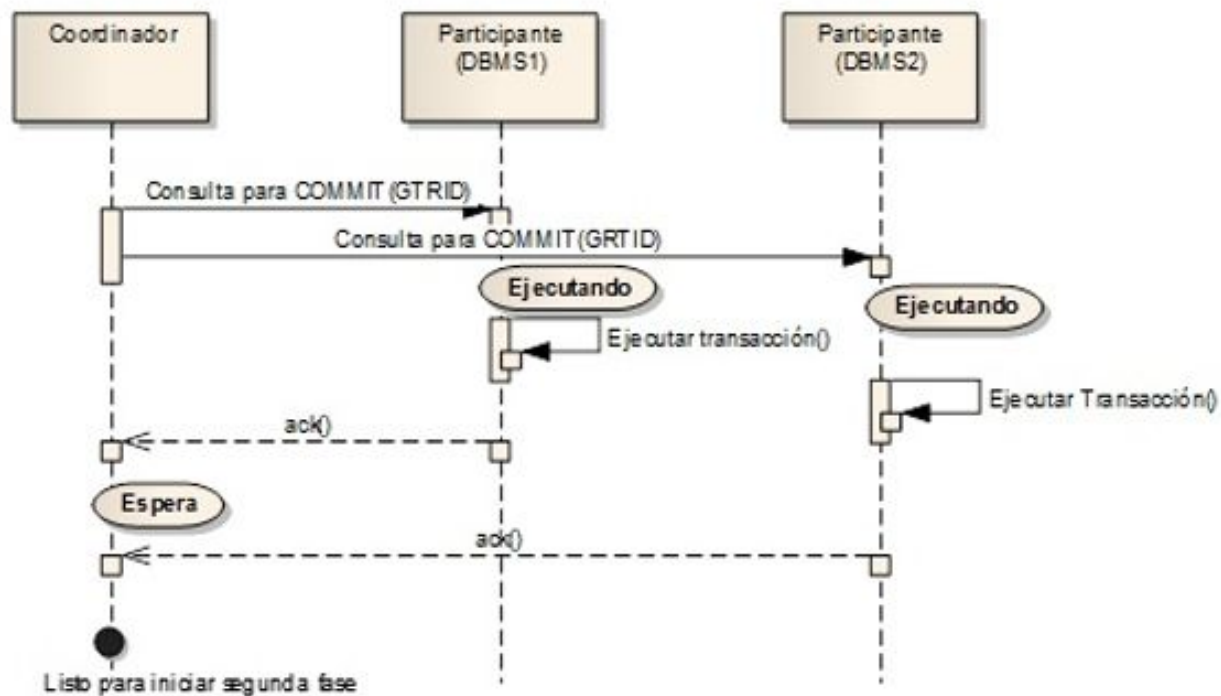
Commit en 2 fases

- Este algoritmo permite que todos los nodos de un sistema distribuido puedan ponerse de acuerdo para confirmar una transacción distribuida.
 - Garantizar las propiedades ACID en un DTP
 - Garantizar que todos los participantes estén sincronizados.
- El Transaction manager asociado con la aplicación que inicia la transacción será llamado el coordinador y las bases de datos involucradas dentro de la transacción global serán llamados los participantes. En Commit en Dos Fases, el coordinador orquesta la sincronización de todos los participantes, para eso implementa dos fases:
 - Primera fase: Fase de Petición del Commit, en donde el coordinador pregunta a todos los participantes si pueden garantizar un commit y los participantes responden. Todos los participantes deben confirmar su actuación.

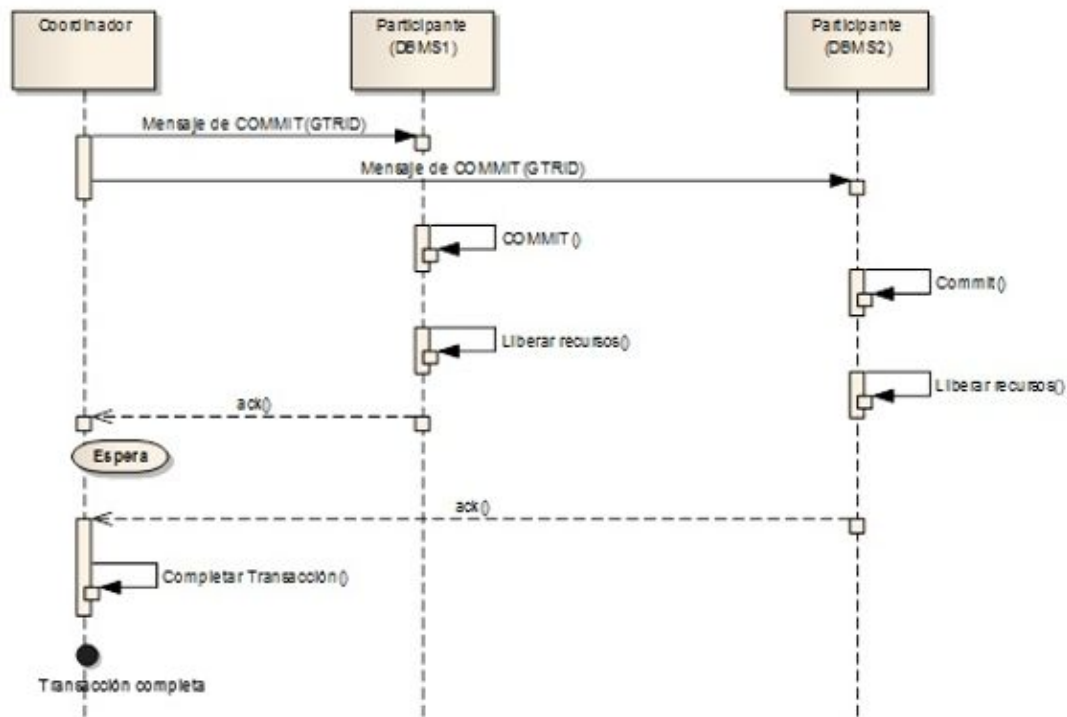
Commit en 2 fases

- Segunda fase: Fase de Commit, en donde el coordinador instruye a todos los participantes a hacer efectivo el commit pedido den la fase anterior (si todos estaban listos), o de abortar la transacción (si alguno de los participantes no estaba listo). Todos los participantes deben confirmar su actuación.

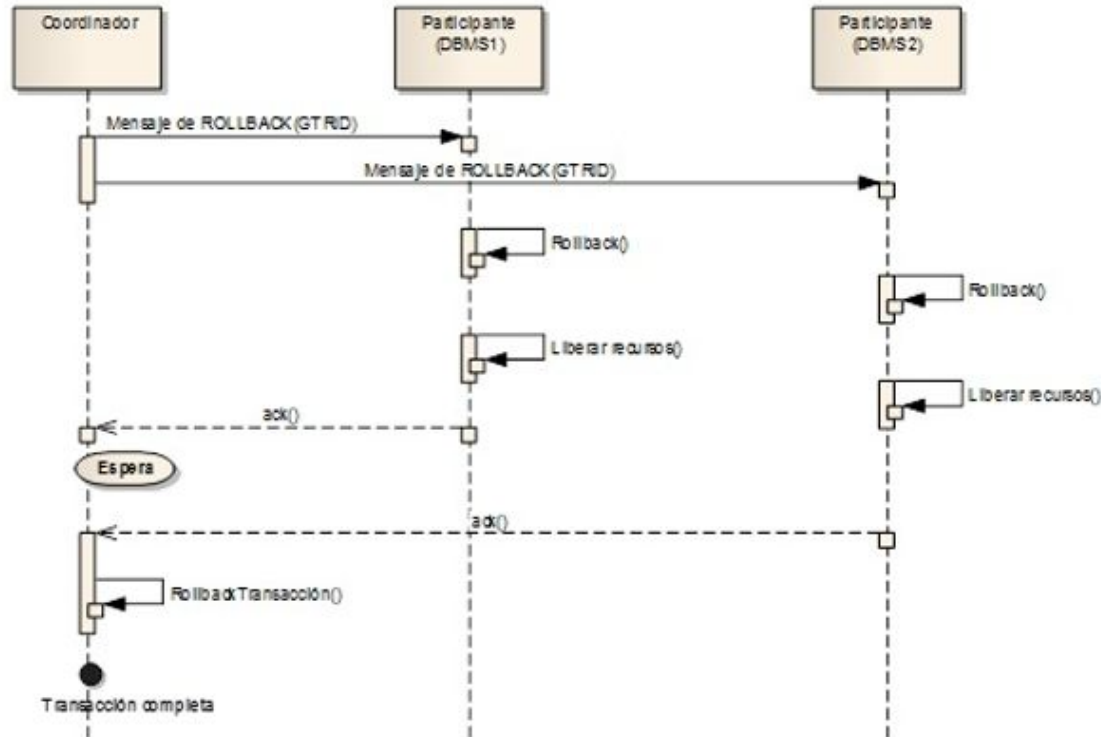
1ra Fase



2da Fase



Si se indica abortar una transacción (part)



Recuperación

- Con el fin de garantizar las propiedades ACID previamente enunciadas, el DBMS necesitará procurarse de un mecanismo de recuperación.
 - Falla de energía o de memoria
 - Error del sistema operativo
 - Excepciones controladas por la aplicación o por los objetos del DBMS que retraen una transacción (rollback)
 - Imposición del Transaction Manager (por ejemplo, el control de deadlocks decide interrumpir una transacción y lanzarla en un intervalo aleatorio posterior de tiempo)
 - Falla de disco
 - Problemas y Catástrofes físicas

Log de Transacciones

- Con el fin de recuperarse ante las fallas anteriormente citadas, los DBMS suelen mantener un log transaccional que le sigue la pista a todas las operaciones que se realizan
- Suele mantenerse en disco y se recomienda su respaldo para protegerlo de los fallos catastróficos.
- Habrá:
 - entradas de inicio de transacción,
 - de fin de las mismas (positivos o negativos),
 - de escritura de recursos, y
 - de un tipo especial, llamado checkpoint.
- Este checkpoint le indica al sistema el momento hasta cuando se han volcado a los archivos físicos de datos las entradas del log.

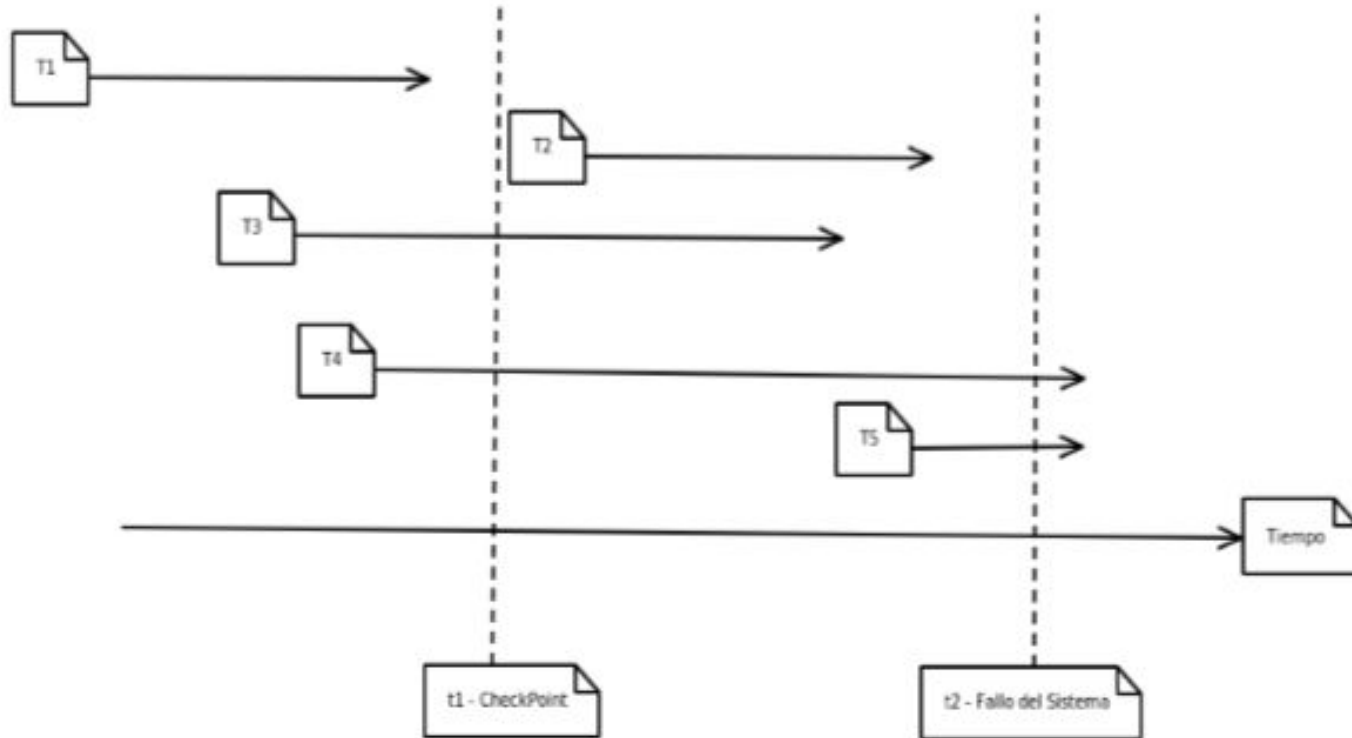
Técnicas de recuperación basadas en actualización diferida

- Al interactuar transaccionalmente con un DBMS no se graban los datos directamente al almacenamiento físico, sino que se escribe en un log al momento de la confirmación de la transacción, y luego de manera diferida a los archivos físicos de datos al momento del checkpoint del log transaccional.
- Si una transacción falla antes de llegar a su confirmación (commit), no hay necesidad de deshacer ninguna operación
 - Una transacción no puede modificar la base de datos antes de llegar a su punto de confirmación (commit)
 - Una transacción no llega a su punto de confirmación sin antes asentar todas sus operaciones de actualización al log transaccional y forzar la escritura (flush) del log al medio (disco).

Técnicas de recuperación basadas en actualización diferida

- El procedimiento para la recuperación, puede describirse de la siguiente manera:
 - El DBMS mantiene dos listas de transacciones:
 - Las transacciones T confirmadas desde el último checkpoint
 - Las transacciones activas T_a
 - Rehacer, a partir del log, todas las operaciones (de escritura) de transacciones confirmadas en el orden en que aparecen en el log.
 - Las T_a que no han sido confirmadas se cancelan efectivamente y deberán reintroducirse.

Técnicas de recuperación basadas en actualización diferida



Técnicas de recuperación basadas en actualización diferida

- Según el procedimiento anteriormente descrito:
 - La transacción T1 se había confirmado antes del último checkpoint (antes de la falla), por lo que no necesita ninguna acción en particular, ya fue escrita a la base de datos.
 - Las transacciones T2 y T3 deben rehacerse, dado que no fueron escritas definitivamente a la base de datos, por lo haber verificado un checkpoint.
 - Las transacciones T4 y T5 se ignoran, nunca recibieron una confirmación.

Técnicas de recuperación basadas en actualización inmediata

- Al interactuar transaccionalmente con un DBMS se graban los datos directamente e inmediatamente al almacenamiento físico.
- Sin embargo, también se utiliza un log transaccional, a fin de que el DBMS sea capaz de recuperarse en caso de fallos.
- Estas variantes hacen uso de un mecanismo de escritura anticipada en el log transaccional.
- En estos casos, las medidas a tomar ante una recuperación tienen que ver con deshacer las escrituras realizadas en la base de datos.
- En estas técnicas, también se asientan checkpoints en los logs.

Técnicas de recuperación basadas en actualización inmediata

- El procedimiento puede describirse de la siguiente manera:
 - El DBMS mantiene dos listas de transacciones:
 - Las transacciones confirmadas desde el último checkpoint (T_c)
 - Transacciones activas (T_a)
 - Deshacer todas las operaciones de T_a en el orden opuesto al que fueron asentadas en el log.
 - Rehacer todas las operaciones de T_c en el orden en que se escribieron en el log

Técnicas de recuperación con Paginación de Sombra

- Esta técnica concibe que la base de datos esté compuesta por varios bloques (páginas) de disco de tamaño fijo para fines de recuperación.
- Tabla de páginas con n entradas
- la i -ésima entrada a la misma apunta a la i -ésima página de la base de datos en el disco.
- Se mantiene en memoria
- Todas las referencias a las páginas de la base de datos en el disco pasan por la tabla de páginas.
- Cuando una transacción comienza a ejecutarse, la tabla de páginas actual se copia a una tabla de páginas sombra, que se guarda en disco mientras la transacción usa la tabla actual.

Técnicas de recuperación con Paginación de Sombra

- Durante la ejecución de la transacción, la tabla de páginas sombra nunca se modifica, cuando se efectúa una operación de escritura, se crea una nueva copia de la página modificada, pero nunca se sobrescribe la copia antigua.
- En vez de ello, la nueva página se escribe en otro sitio.
- En el caso de las páginas actualizadas por las transacciones, se mantienen dos versiones. La tabla de páginas sombra hace referencia a la versión antigua, y la tabla de páginas actuales a la nueva.
- La recuperación de una falla mientras se ejecuta una transacción es trivial: Se liberan las páginas modificadas de la base de datos y se deshace la tabla de páginas actuales. El estado consistente anterior está disponible a través de este mecanismo.

Técnicas de recuperación con Paginación de Sombra

- En el momento de la confirmación de una transacción, se desechan las páginas de sombras previas y liberar las páginas antiguas.
- Las ventajas de este método pasan por la facilidad de recuperación, mientras que las desventajas apuntan al costo de mantener una cantidad de páginas extra que no se mantienen con otras técnicas.

Esquema para una recuperación catastrófica

- Back up Full 1
 - Back up Diferencial 1.1
 - Back up Diferencial 1.2
 - Back up Diferencial 1.n
- Log transaccional



Bases de Datos I

Clase 11

Otimización

Optimización

- ¿Cuándo se empieza a optimizar?

Metas de Optimización

- Sistemas OLTP

- Throughput:

- trx/t

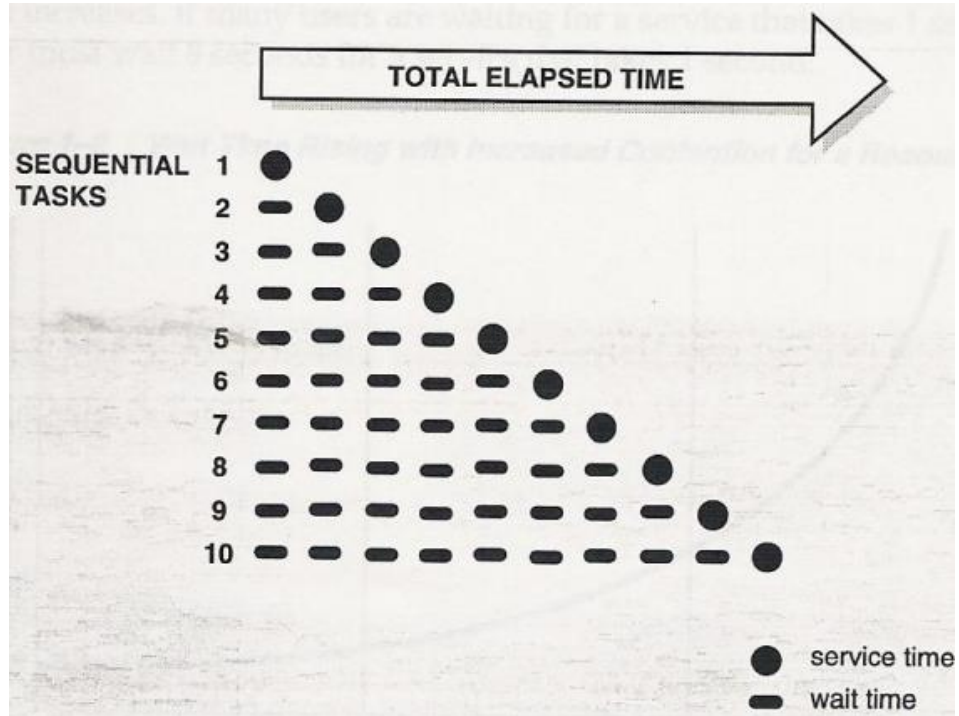
- Sistemas OLAP

- Tiempo de Respuesta

¿Cómo optimizar tiempo de Respuesta?

- $Tr = ts + te$
 - Tiempo de Servicio
 - Tiempo de Espera
- O bien
 - Reducimos tiempo de servicio
 - Reducimos tiempo de espera

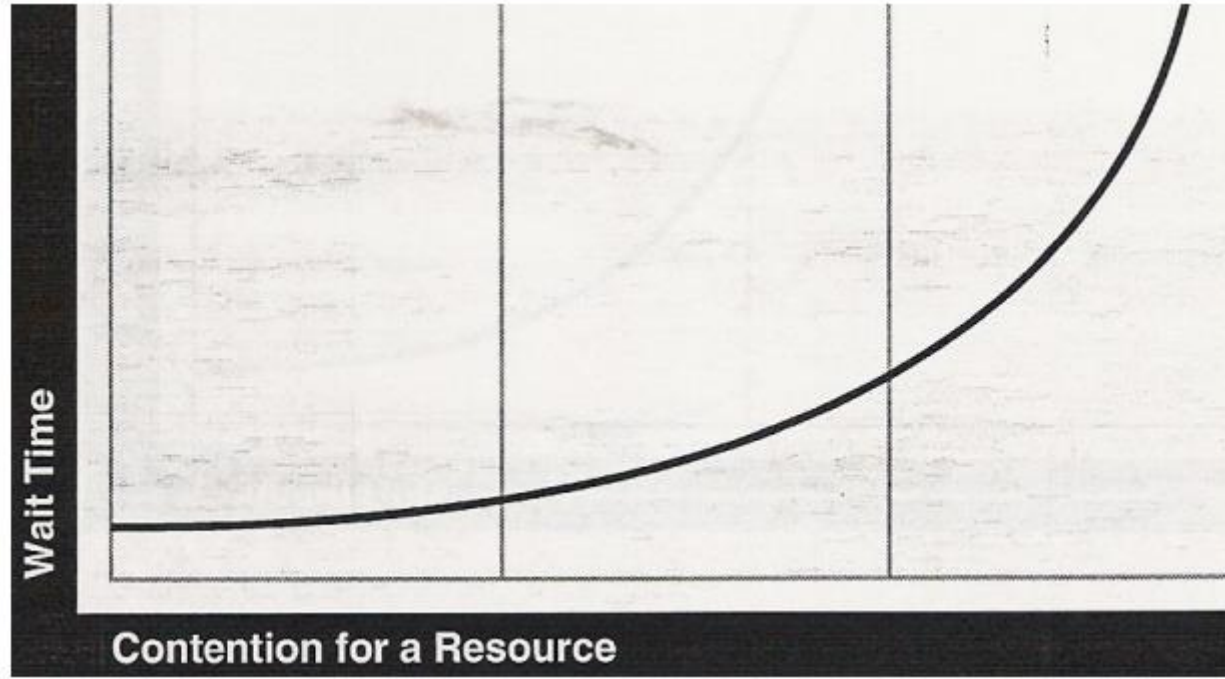
Tiempo de Respuesta



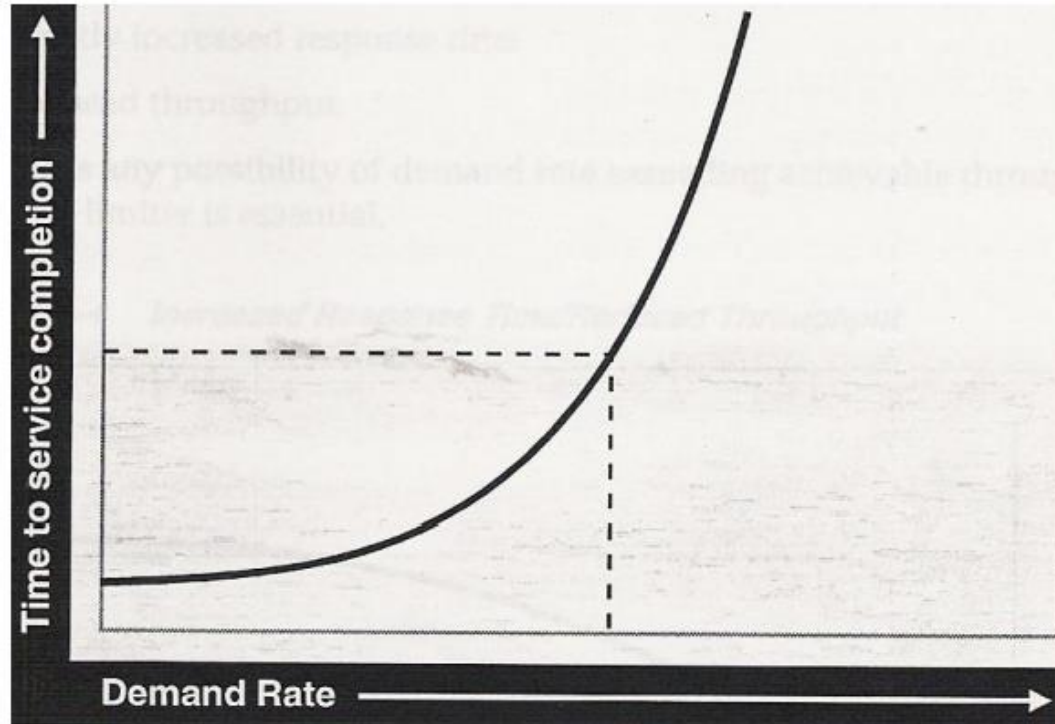
¿Cómo optimizar Throughput?

- Cantidad de trabajo realizado en una cantidad de tiempo dada.
- $ts = w/t$
- Se puede:
 - Hacer más trabajo con los mismo recursos (reducir el tiempo de servicio)
 - Hacer el trabajo más rápido, reduciendo el tiempo de respuesta general.
 - Para hacer esto, hay que prestar atención al tiempo de espera. Por ejemplo, agregando 1 CPU.

Tiempo de Espera



Tiempo de Servicio



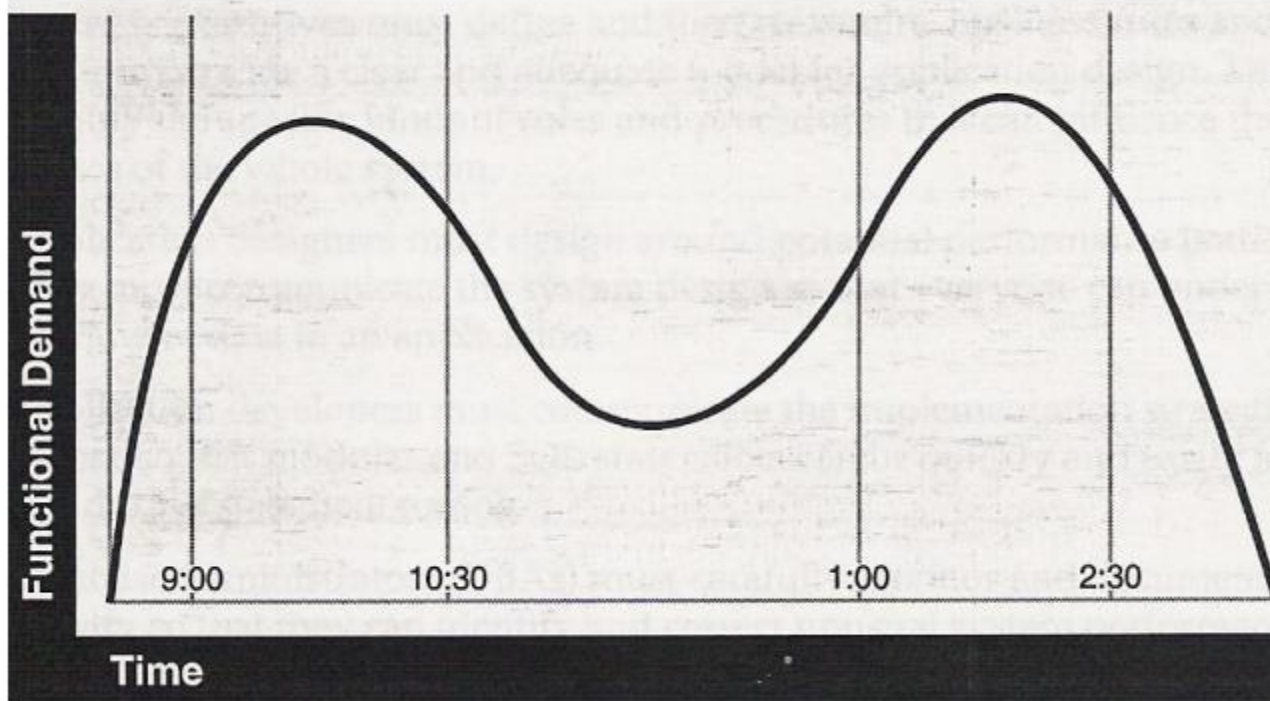
Throughput



Ajustes básicos para problemas de performance

- Ajustar el consumo unitario:
 - Usando menos recursos por transacción o reduciendo el tiempo de servicio.
 - También pueden reducirse el número de operaciones de I/O por transacción.
- Ajustar la demanda funcional:
 - Re planificando o redistribuyendo el trabajo.
- Ajustando la capacidad:
 - Incrementando o re-allocando recursos

Ajustes básicos para problemas de performance



Reglas o Costos

- Existen dos técnicas fundamentales para optimizar consultas que puede utilizar un optimizador:
 - Reglas:
 - Se basa en reglas heurísticas para ordenar las operaciones para resolver la consulta.
 - Costo:
 - Se basa en la estimación sistemática del costo de diferentes estrategias de ejecución

Selectividad de una condición atómica

- Se define cómo la razón entre la cantidad de tuplas (o registros de ahora en más) que satisfacen la condición, y el número total de registros de la tabla (relación).

Algoritmos básicos para la ejecución de consultas

- Selección

- Búsqueda lineal: por fuerza bruta se evalúan todas la tuplas.
- Búsqueda binaria: si el archivo está ordenado por el criterio por el cual se quiere seleccionar y el criterio es una igualdad, se utiliza este tipo de búsqueda.
- Uso de una PK: si el criterio implica una comparación de igualdad sobre un atributo que es clave primaria (o que pertenece a la clave primaria y es el primero en ella), se utiliza el índice, reduciendo la complejidad a la indicada por el índice.
- Uso de una PK para múltiples registros: si el criterio implica una comparación de $\leq$, $>$ o $\geq$, igualdad sobre un atributo que es clave primaria (o que pertenece a la clave primaria y es el primero en ella), se utiliza el índice por rangos, reduciendo la complejidad.

Algoritmos básicos para la ejecución de consultas

- Uso de un índice secundario: Es el mismo caso que los dos anteriores, pero si los criterios se aplican sobre un atributo que tenga específicamente creado un índice.
- Si la condición es conjuntiva (dos o más condiciones unidas por AND), también se pueden utilizar los siguientes métodos:
 - Selección conjuntiva: Si una de las conjunciones puede resolverse vía los métodos anteriores (salvo búsqueda lineal), primero se resuelve esa conjunción y luego sobre el conjunto resultante se emplea la misma técnica hasta que queden resueltas las conjunciones o haya que utilizar el método de búsqueda lineal
 - Selección conjuntiva con un índice compuesto: Si 2 o más de las conjunciones matchea con la definición de un índice compuesto, se utilizará ese índice. Luego se continuará resolviendo las conjunciones sucesivamente.

Algoritmos básicos para la ejecución de consultas

- Selección conjuntiva por intersección de punteros a registros: Si se dispone de índices secundarios (que apliquen a la consulta) que utilicen punteros a registros (no a bloques), en los casos que el criterio sea de igualdad, se pueden usar dichos índices para obtener los punteros a registros para cada criterio, de modo que se pueda aplicar la intersección entre esos conjunto de punteros para determinar cuáles son los registros válidos.

Algoritmos básicos para la ejecución de consultas

- Join

- Nested Loops (ciclos anidados): Por fuerza bruta, por cada registro de uno de los conjuntos, se evaluarán todos los registros del otro. De ese modo la cantidad de registros a evaluar es el producto de las cantidades de registros de cada conjunto que participa en el join.
- Index Nested Loops: Similar al anterior, sólo que puede utilizar un índice.
- Sort-Merge Join: Si las dos tablas del join tienen un índice clusterizado por el atributo de junta, se podrá implementar el sort-merge join de la forma más eficiente:
 - Se examinan ambos archivos en orden, buscando los registros que tienen los mismos valores en ambas tablas, de este modo se logra un apareo por el cuál un registro es examinado una sola vez para compararlo con el otro.

Algoritmos básicos para la ejecución de consultas

- Hash-Join: Los registros de ambas tablas se dispersan al mismo archivo de dispersión, usando la misma función de hash, con los atributos de join como claves del hash.
 - Una sola pasada por la tabla con el menor número de registros dispersa sus registros a buckets del archivo de dispersión.
 - Una sola pasada por el otro archivo dispersa entonces sus registros a la cubeta apropiada, donde el registro se combina con todos los registros coincidentes de la primera tabla.

Algoritmos básicos para la ejecución de consultas

- Proyección

- Es la más simple de implementar, ya que si no es necesario eliminar duplicados, se traerá la lista de atributos necesarios.

- Operaciones de manejo de conjuntos:

- Producto cartesiano: Es una de las operaciones más caras de realizar, ya que se deberán traer $T_1 \times T_2$ registros
- Unión: Adicionalmente al costo de evaluar las dos consultas que une la sentencia UNION, se hará una pasada extra por ambos conjuntos, tanto para retraerlos, cómo para eliminar duplicados.
- Unión Aditiva: Igual que la unión, sólo que no se eliminan duplicados.

Algoritmos básicos para la ejecución de consultas

- Intersección: Se utiliza un esquema similar al del Hash-join, se dispersa el conjunto 1 y luego al dispersarse el conjunto 2 (teniendo en cuenta todos los atributos siempre), sólo se tomarán los coincidentes.
- Diferencia de conjuntos: De forma similar a la intersección, se dispersarán ambos conjuntos, sólo que en vez de tomar los coincidentes en la segunda pasada, se los eliminará.

Estimaciones de costo

- El costo de ejecutar una consulta consiste en:
 - Costo de acceso a almacenamiento secundario
 - Costo de almacenamiento de archivos intermedios
 - Costo de cómputo
 - Costo de comunicación

Información en el catálogo para estimación de costo

- Número de registros por tabla
- Número de bloques
- Estimaciones aproximadas de los dos anteriores cuándo no se pueda disponer del número exacto.
- Método de acceso primario y atributos de acceso primario
- Atributos de acceso secundario
- Si existe índice tipo cluster
- Número de niveles de cada índice multinivel
- Número de valores distintos de un atributo indexado (cardinalidad de la selección o número medio de registros que satisfarán una condición de selección sobre dicho atributo)

Funciones de Costo para selección

1. Enfoque de búsqueda lineal: por fuerza bruta, examina todos los registros/bloques del archivo para obtener los que satisfagan la condición de selección, por lo tanto $C(\text{sel})=r$. Si la condición es una igualdad, en promedio se evaluarán la mitad de los registros, o sea $C(\text{sel})=r/2$.
2. Búsqueda binaria: Aproximadamente $C(\text{sel})=\text{Log}_2 r$.
3. Empleo de un índice primario para obtener un sólo registro: $C(\text{sel})= x + 1$ (dónde x es el número de niveles). Si se trata de un hash, $C(\text{sel}) = 1$.
4. Empleo de un índice para obtener múltiples valores (rangos): Aproximadamente en promedio la mitad de los registros satisfarán la condición, por lo que $C(\text{sel})= x + r/2$.
5. Empleo de un índice clusterizado para obtener múltiples registros: Suponiendo que la condición de igualdad satisface unos s registros (cardinalidad de la condición de selección), $C(\text{sel})= c + x$.
6. Empleo de un índice árbol B*: si la condición es de igualdad, y la selectividad de la condición es s , $C(\text{sel}) = x + s$. Si la condición es de rango, $C(\text{sel}) = x + r/2$.

Funciones de costo para join

Selektividad del Join: $sj = | (R \text{ join } S) | / | (R \times S) | = | (R \text{ join } S) | / (|R| \cdot |S|)$

$$0 \leq sj \leq 1$$

De este modo el optimizador puede estimar a priori el tamaño del resultado de un join vía la fórmula:

$$| R \text{ join } S | = sj \cdot |R| \cdot |S|$$

- Nested Loops: suponiendo que se utiliza R para el ciclo superior (o externo), $C(\text{join}) = |R| + |R| \cdot |S| + (sj \cdot |R| \cdot |S|)$. El último término de la fórmula es el costo de escritura en almacenamiento temporario.
- Empleo de índices: Si hay un índice para el atributo de condición de reunión en S, podemos tomar todos los registros de R y utilizar el índice en S: $C(\text{join}) = |R| + (|R| \cdot (x + sb)) + (sj \cdot |R| \cdot |S|)$ donde x es la cantidad de niveles del índice y sb la cardinalidad de selección sobre la condición de reunión.. En el caso de acceder por un índice primario, sb se convierte en 1. Si hay un hash para una de las dos tablas sobre el atributo de reunión, se reemplaza $(x + sb)$ por h, donde h es el número medio de accesos necesarios para obtener un registro.
- Merge-Sort: si los archivos están ordenados sobre el atributo de reunión, el costo es $C(\text{join}) = |R| + |S| + (sj \cdot |R| \cdot |S|)$. Ahora si hay que ordenar los archivos, se le agrega el costo aproximado, es decir, el costo queda como $C(\text{join}) = |R| + |S| + (sj \cdot |R| \cdot |S|) + (|R| \cdot \log_2 |R|) + (|S| \cdot \log_2 |S|)$

Metodología para optimizar una base de datos

- Determinar cuáles son los queries que consumen la mayoría de los recursos
 - Los que consumen grandes cantidades de recursos
 - Los que consumen grandes cantidades de recursos por registro
 - Los más ejecutados
- Optimizar esos queries para que utilicen menos recursos
 - Tratando de que utilicen menos recursos cuándo son ejecutados
 - O usando dichos comandos menos frecuentemente.

Enfoques para la optimización de comandos SQL

- Reestructurar los índices:
 - Esta actividad es siempre un buen punto de partida, ya que es de gran impacto en la performance.
 - No hay que creer que los índices son la panacea, ya que un índice bien creado la mejorará, pero 50 índices sobre una tabla tienen muy poca probabilidad de mejorarla.
 - Eliminar índices innecesarios
 - Indexar los accesos críticos
 - Considerar cluster teniendo en cuenta la unicidad

Enfoques para la optimización de comandos SQL

- Reestructurar los comandos SQL:
 - Después de reestructurar los índices, se puede pasar a la reestructuración de los comandos.
 - Muchas veces es más fácil reescribir SQL que repararlo.
 - Probablemente si se sabe la esencia del comando, se podrá escribir otro equivalente.
 - Se pueden utilizar los resultados del comando Explain plan para comparar los planes de ejecución y costos de dos o más comandos para determinar cuál es el más eficiente.
 - Ver Apunte

Enfoques para la optimización de comandos SQL

- Reestructurar los datos: Luego de realizar los dos pasos anteriores, puede considerarse la reestructuración de los datos:
 - Introducir valores derivados al esquema, evitando los comandos GROUP BY en código que necesita una pronta respuesta. (denormalizar)
 - Implementar entidades faltantes y sus tablas de intersección
 - Reducir la carga de la red:
 - i. Migrando
 - ii. Replicando
 - iii. Particionando
 - iv. Distribuyendo

Seguridad en RDBMS



Temas



Seguridad e Integridad



Seguridad informática: dos enfoques



DBMS y Seguridad Informática



Control Discrecional (DAC)



DCL: GRANT y REVOKE



Vistas y Seguridad



Seguridad y Stored Procedures



Roles



Grupos o Perfiles



Seguridad e Integridad

¿Son lo mismo?

- **Seguridad** significa proteger los datos contra usuarios no autorizados. O asegurar que los usuarios están autorizados para hacer las cosas que intentan hacer.
- **Integridad** significa proteger los datos de usuarios autorizados. O asegurar que los usuarios hacen lo correcto con los datos.

Similitudes: en ambos casos el sistema debe ser consciente de algunas **limitaciones (constraints)** que los usuarios no deben violar.

Esos constraints deberán ser especificados en un lenguaje adecuado, y deben resguardarse en el catálogo del sistema.

El DBMS debe supervisar las operaciones de los usuarios con el fin de garantizar que los constraints son observados.



Seguridad Informática: dos enfoques

Control Discrecional (DAC)

Discretionary Access Control es una forma de acceso a recursos basada en los propietarios y grupos a los que pertenece un objeto. Se dice que es discrecional en el sentido de que un sujeto puede transmitir sus permisos a otro sujeto.

Control mandatorio (MAC)

Mandatory Access Control se basa en políticas. Existen un conjunto de reglas de autorización (políticas) las cuales determinan si una operación sobre un objeto realizada por un sujeto esta o no permitida basándose en los atributos de ambos. En este caso, el Mandatory refleja el hecho de que las políticas están centralizadas y no pueden ser sobrescritas por un sujeto.



Seguridad Informática: dos enfoques

En **DAC** el acceso esta descentralizado, siendo el propietario de cada objeto el encargado de asignar los permisos de los diversos sujetos que accederán a ellas. En cambio con el MAC los objetos y los sujetos tan solo tienen atributos, pero son las políticas las que se encargan de autorizar o denegar una acción.

MAC es un sistema centralizado, en el cual las decisiones de seguridad no recaen en el propietario de un objeto y es el sistema el que fuerza el cumplimiento de las políticas por encima de las decisiones de los sujetos.

Granularidad: en ambos casos, la unidad de datos u “objeto de datos” a ser protegida puede ir desde la base completa hasta un componente específico dentro de una tupla específica.



DBMS y Seguridad Informática

Independientemente del tipo de control que se ejerza, las decisiones de qué usuarios están autorizados a qué información son de tipo político y no técnico; **lo único que puede hacer el DBMS es forzar a que esas decisiones se cumplan.**

- El resultado de las decisiones políticas debe ser conocido y recordado por el sistema. Esto se logra por medio de sentencias de definición en algún lenguaje apropiado que crea un equivalente de la decisión en el catálogo.
- Dado un requerimiento de acceso debe haber un medio para chequear la información de seguridad almacenada en el catálogo. El requerimiento de acceso estará compuesto por la operación requerida, el objeto involucrado y el usuario que origina el pedido. Este chequeo lo realizará el subsistema de seguridad del DBMS también conocido como subsistema de autorizaciones.



DBMS y Seguridad Informática

El enfoque básico de la seguridad y las autorizaciones que tomaron los proveedores de DBMSs para el acceso seguro a las bases de datos es que todos los recursos sean controlados por el DBMS. **No existen autorizaciones de default** para los usuarios que se conecten al motor.

Por lo tanto para que un usuario pueda ejecutar alguna operación debe darse alguna de las siguientes condiciones:

- Al usuario se le otorgó la posibilidad de ejecutar la operación.
- La operación fue otorgada genéricamente a todos los usuarios.



Control Discrecional (DAC)

El SQL estándar actual soporta solo el control de acceso discrecional.

Elementos de SQL involucrados:

- El mecanismo de las **Vistas** que puede ser usado para ocultar datos sensitivos a usuarios no autorizados.
- Los **Stored Procedures** que permiten operar sobre las tablas base sin autorizarlas explícitamente.
- El **Subsistema de Autorizaciones** en sí mismo que permite a los usuarios armar un esquema de seguridad selectivo y dinámico.



DCL

El control de seguridad y las autorizaciones del DBMS se llevan a cabo con el **Data Control Language (DCL)**.

Las sentencias de DCL se utilizan para controlar qué usuarios tienen acceso a qué objetos y comandos. Existen dos tipos:

- **GRANT** que asigna permisos a un usuario.
- **REVOKE** que remueve los permisos de un usuario.

Importante: El creador de cualquier objeto obtiene automáticamente todos los privilegios, que tengan sentido, sobre ese objeto. Ejemplo:

El creador de la tabla T tendrá inmediatamente la posibilidad de hacer SELECT, INSERT, UPDATE, DELETE, etc. sobre T. Y puede asignarle esos permisos a otros usuarios.

GRANT

Para usar la sentencia **GRANT**, el usuario debe ser:

- El dueño (owner) del objeto a ser autorizado
- O tener la autoridad para hacerlo por pertenecer a un grupo de usuarios especiales
- O tener la posibilidad de hacerlo debido a que se le otorgaron permisos con la opción "WITH GRANT OPTION"

La sintaxis básica de la sentencia es la siguiente:

GRANT *<Lista de privilegios separados por comas>*

ON *<Objeto>*

TO *<Lista de id de usuarios separados por comas>*

[WITH GRANT OPTION]

INSERT, DELETE, SELECT, etc.

Tabla, Vista, etc.

ID de Usuario, PUBLIC, etc.

Atributo especial del comando GRANT



GRANT

- Los **<privilegios>** válidos son: USAGE, SELECT, INSERT, UPDATE, DELETE y REFERENCE.
- Los **<objetos>** válidos son: DOMAIN <nombre de dominio> y TABLE <nombre de la tabla>. TABLE es opcional y puede hacer referencia a una vista.
- La **<Lista de id de usuarios separados por comas>** puede reemplazarse por la palabra reservada PUBLIC, para indicar a todos los usuarios definidos en el sistema. Al utilizarse PUBLIC no podrá usarse la opción WITH GRANT OPTION.
- **“WITH GRANT OPTION”** permite a los usuarios pasar la autoridad de otorgar el privilegio a otros. Puede administrarse de dos maneras:
 - Descentralizado: Fácil de establecer, más difícil de controlar. A medida que más y más usuarios obtienen la autoridad para conceder privilegios, el alcance de la autoridad se amplía y es difícil de manejar.
 - Centralizado: Es generalmente más fácil de administrar, pero carga al administrador por ser el único en manejar los privilegios en el entorno.

REVOKE

Si el usuario A otorga algún privilegio al usuario B, el usuario A puede quitar tal privilegio del usuario B. Para quitarlo se utiliza la sentencia **REVOKE**.

La sintaxis básica de la sentencia es la siguiente:

```
REVOKE [GRANT OPTION FOR] <Lista de privilegios separados por comas>  
ON <Objeto>  
FROM <Lista de id de usuarios separados por comas>  
<Opción>
```

*INSERT, DELETE, SELECT,
CONNECT, EXECUTE, etc.*

Tabla, Vista, etc.

ID de Usuario

CASCADE, RESTRICT

REVOKE

- ***“GRANT OPTION FOR”*** significa que se quitará la posibilidad de propagar la autorización a otros usuarios.
- ***<Lista de privilegios separados por comas>, <Objeto> y <Lista de id de usuarios separados por comas>*** son iguales a los campos del GRANT.
- ***<opción>*** es RESTRIC o CASCADE.

Ejemplo:

Supongamos que P es un privilegio de algún objeto y supongamos que un usuario A otorga P al usuario B, que a su vez le otorga P al usuario C.

¿Qué sucederá si quita P al usuario B?

El privilegio P en manos de C quedaría “abandonado” (porque deriva de B que ya no existe).

Dos opciones:

RESTRICT: provoca que REVOKE falle si el usuario B tiene permisos otorgados.

CASCADE: hace que los privilegios de B y los que haya otorgado caigan.

Vistas y Seguridad

El mecanismo de Vistas permite que la base sea conceptualmente dividida en partes de modo que la información sensible pueda ocultarse a usuarios no autorizados.

Pero, NO permite especificar las operaciones que se les permite ejecutar a los usuarios autorizados sobre cada una de las partes. Esta tarea le queda a la sentencia GRANT.

Ejemplo:

```
CREATE VIEW SOLOBSAS AS  
  SELECT RAZÓN _ SOCIAL, LOCALIDAD, TELEFONO  
  FROM COMERCIOS  
  WHERE PROVINCIA='BUENOS AIRES'
```

La vista define los datos sobre los que operará la autorización.

Una sentencia GRANT
podría ser:

```
GRANT SELECT  
  ON SOLOBSAS  
  TO USUARIO_A, USUARIO_B
```

Vistas y Seguridad

Hay vistas que modifican su resultado dependiendo del contexto en el que operan.

Ejemplo:

```
CREATE VIEW SEGÚN_USER AS  
  SELECT APELLIDO, NOMBRE, EDAD  
  FROM CLIENTES  
  WHERE PRESENTADO_POR=CURRENT USER
```

La vista anterior devolverá diferentes resultados dependiendo de qué usuario ejecute la consulta. Otorgar permisos a esta vista significaría que cada usuario puede ver la información de aquellos clientes que fueron presentados por él.



Seguridad y Stored Procedures

Los **Stored procedures** (SP) pueden usarse para proveer un nivel adicional de seguridad.

Un SP es un programa (o procedimiento) almacenado físicamente en una base de datos y ejecutado directamente en el motor de bases de datos, el cual usualmente corre en un servidor separado.

El privilegio de ejecutar un SP debe ser explícitamente otorgado o quitado independientemente de la seguridad implementada en las tablas base.

Los SP pueden codificarse para que accedan a determinadas filas y columnas. Puede otorgarse a un usuario la posibilidad de ejecutar un SP y, aunque no tenga la posibilidad de acceder a las tablas base éste tendrá posibilidad de operar sobre ellas a través de la lógica del SP generando así un nivel extra de seguridad.



Roles y Grupos de Autorización

Además de otorgar privilegios a usuarios individuales, los DBMSs pueden tener la capacidad de otorgar:

- Privilegios específicos a un Rol, que a su vez es otorgado a otros usuarios.
- Grupos de privilegios predefinidos a los usuarios.

La terminología depende del DBMS, algunos llaman Grupos a los Roles y viceversa.



Roles

Una vez definido un Rol, este puede ser usado para otorgar uno o más privilegios preasignados a un usuario. Un rol es esencialmente una **colección de privilegios**.

Ejemplo:

```
CREATE role MANAGER;  
COMMIT;  
GRANT select, insert, update, delete on employee to MANAGER;  
GRANT select, insert, update, delete on job_title to MANAGER;  
GRANT execute on payroll to MANAGER;  
COMMIT;  
GRANT MANAGER to user1;  
COMMIT;
```



Grupos o Perfiles

La autoridad a nivel de grupo/perfil es similar a los roles. Sin embargo, cada DBMS provee sus grupos/perfiles predefinidos y no pueden ser cambiados.

Grupos/Perfiles comunes a la mayoría de los DBMS:

- **System administrator:** SA o SYSADM, es el más poderoso dentro del DBMS. Puede ejecutar todos los comandos y acceder a todas las bases y objetos. Es usualmente el responsable de la instalación del DBMS.
- **Database administrator:** DBADM o DBA, tiene todos privilegios sobre una base específica, además de la posibilidad de acceder, pero no modificar, los datos en las tablas dentro de la base. Los usuarios asignados a este perfil puede dropear y alterar cualquier objeto dentro de la base.



Grupos o Perfiles (cont.)

- **Database maintenance:** DBMAINT, incluye los privilegios para el mantenimiento de los objetos de la base (tales como la posibilidad de ejecutar utilitarios y comandos) . Al igual que el grupo/perfil DBA, este privilegio se otorga por cada base.
- **Security administrator:** SSO, la posibilidad de otorgar privilegios dentro del DBMS usando las sentencias GRANT y REVOKE. Puede ejecutar cualquier otra actividad relacionada con la seguridad, incluyendo la administración del login/password, auditoria, el diseño de la configuración de seguridad.
- **Operations control:** OPER o SYSOPR, el controlador de operaciones tiene la autoridad de ejecutar tareas operativas tales como backup y recovery, o cancelar tareas que no estén trabajando correctamente.